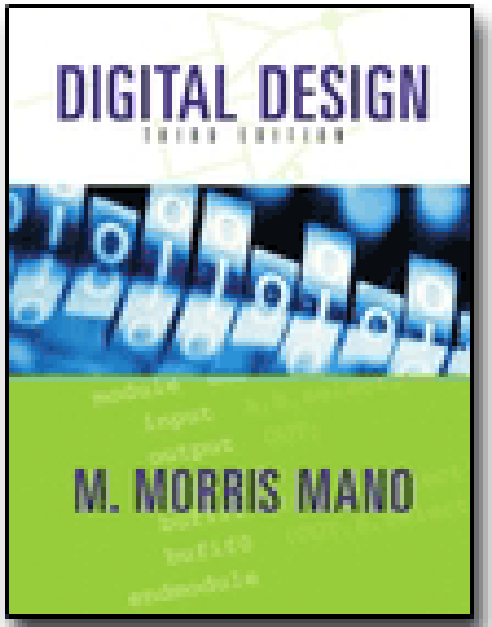
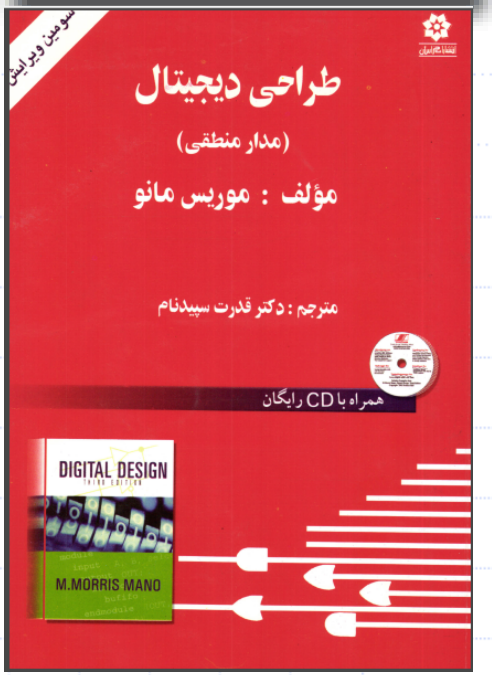


بسم اللّه الرحمن





مدارهاي منطقي

مرجع: مدارهاي منطقي ديجيتال

نوشته: مانو – مترجم : دکتر سپ

مدرس: وحیده منعمی زاده

دانشگاه صنعتی سجاد

فصل اول:

ورود به سیستم دیجیتال

سیستم دودویی اعداد(binary):

- آسان برای کامپیوتر ها, ناملموس برای ما
- از ارقام دودویی((binary digits (bits)، به جای ارقام ده دهی استفاده می کند.
- n بیت داده شده می تواند نشانگر 2^n عدد باشد.
- با ده انگشت می شود تا ۱۰۲۳ شمرد!
- در این سیستم نیز از موقعیت، وزن را تعیین می کند.

Base B : 0 ≤ digit ≤ B -1

Base 10 : 0 ≤ digit ≤ 9 (10 -1)
Base 2 : 0 ≤ digit ≤ 1 (2-1)
Base 8 : 0 ≤ digit ≤ 7 (8 -1)
Base 16 : 0 ≤ digit ≤ 15 (16 -1)

Hexadecimal	Octal	Binary	Decimal
0	0	0	0
1	1	1	1
2	2		2
3	3		3
4	4		4
5	5		5
6	6		6
7	7		7
8			8
9			9
A (decimal value of 10)			
B (decimal value of 11)			
C (decimal value of 12)			
D (decimal value of 13)			
E (decimal value of 14)			
F (decimal value of 15)			

تبدیل از مبنای دو به مبنای ده

1 2 4 8 16 32

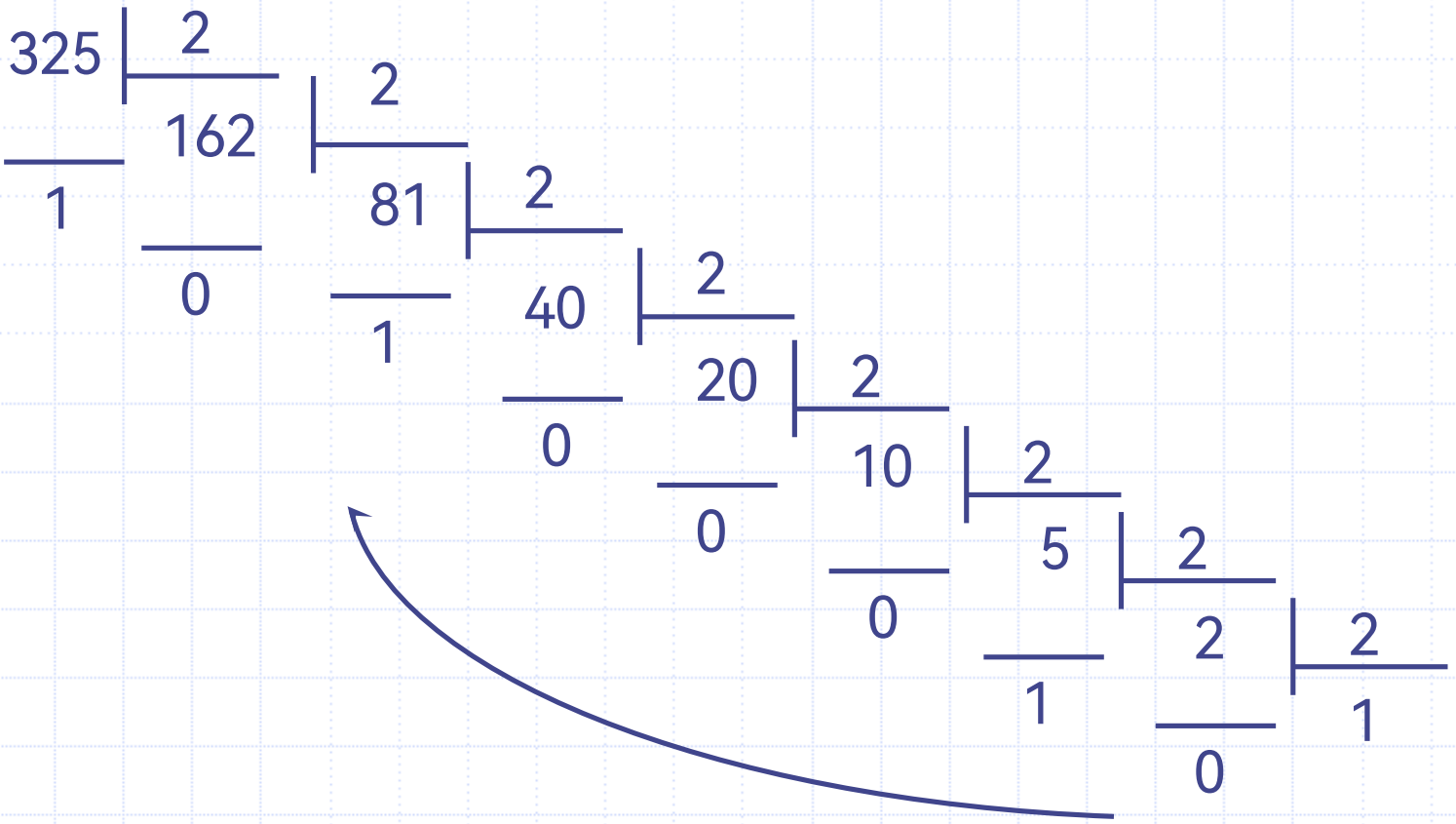
$(1\ 0\ 1\ 1\ 1\ 0)_2$ $= 0 \times 1 + 1 \times 2 + 1 \times 4 + 1 \times 8 + 0 \times 16 + 1 \times 32 = (46)_{10}$

$2^5\ 2^4\ 2^3\ 2^2\ 2^1\ 2^0$

تبدیل از مبنای ده به مبنای دو

روش اول : تقسیمات متوالی

$(325)_{10} \rightarrow (101000101)_2$



تبدیل از مبنای B به مبنای 10

- مثال

$$(a_{n-1}\ a_{n-2}\ \dots\ a_0 . a_{-1} \dots a_{-m})_B = (N)_{10}$$

$$N = (a_{n-1} * B^{n-1}) + (a_{n-2} * B^{n-2}) + \dots + (a_0 * B^{0}) + (a_{-1} * B^{-1}) + \dots + (a_{-m} * B^{-m})$$

- $(4173)_{10} = (4 * 10^3) + (1 * 10^2) + (7*10^1) + (3*10^0) = (4173)_{10}$
- $(0756)_8 = (0 * 8^3) + (7 * 8^2) + (5*8^1) + (6*8^0) = (494)_{10}$
- $(3b2)_{16} = (3 * 16^2) + (11*16^1) + (2*16^0) = (946)_{10}$
- $(2E6)_{16} = ?$
- $(2E6)_{16} = (2 * 16^2) + (14*16^1) + (6*16^0) = (?)_{10}$

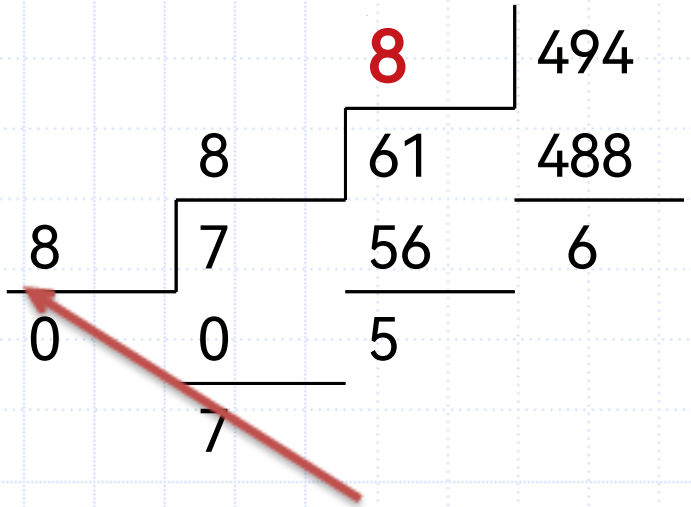
تبدیل از مبنای 10 به مبنای B

- $(N)_{10} = (\underbrace{a_{n-1} \quad a_{n-2} \quad \dots \quad a_0}_{\text{Integer part}})_B$

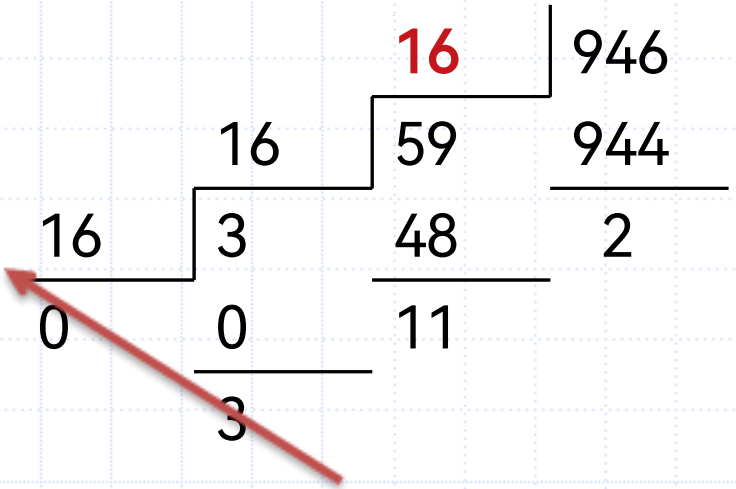
قسمت صحیح را به Base-B تبدیل کنید (با تقسیمات متوالی)

25

تبدیل قسمت صحیح به مبنای B



$(494)_{10} = (756)_8$



$(946)_{10} = (3B2)_{16}$

تبدیل باینری و اکتال

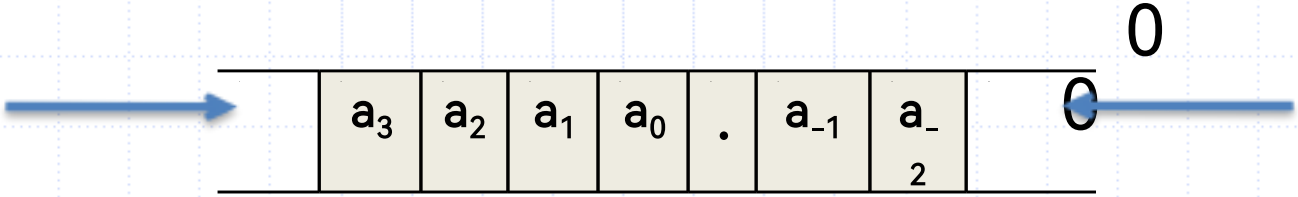
Octal به Binaryy

Octal	0	1	2	3	4	5	6	7
Binary	000	001	010	011	100	101	110	111

$2^3 = 8$

– هر رقم در قالب اکتال: 3 رقم در قالب باینری

– اگر تعداد رقم ها توسط 3 قابل تقسیم نباشد، صفرهای اضافی اضافه می شود:



- 43 = 043 = 000043 = 043.0 = 043.000
- $(10011)_2 = (010011)_2 = (23)_8$

2 3

تبدیل باینری و اکتال

- Binaryyبه Octal
  - هر رقمی را با 3 رقم باینری جایگزین کنید

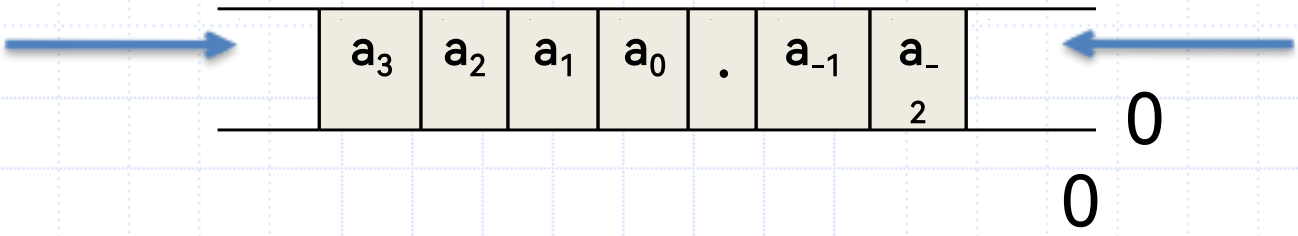
- $(5)_8 = (101)_2$
- $(1)_8 = (001)_2$
- $(51)_8 = (101\ 001)_2$
- $(23)_8 = (010\ 011)_2$

تبدیل باینری و هگزادسیمال

Binaryy به Hexadecimal

$2^4 = 16$

- هر رقم در قالب هگزادسیمال: 4 رقم در قالب باینری
- اگر تعداد رقم ها به 4 تقسیم نمی شود، صفرهای اضافی اضافه می شود:



- $(1111101)_2 = (01111101)_2 = (7D)_{16}$

7 D

تبدیل باینری و هگزادسیمال

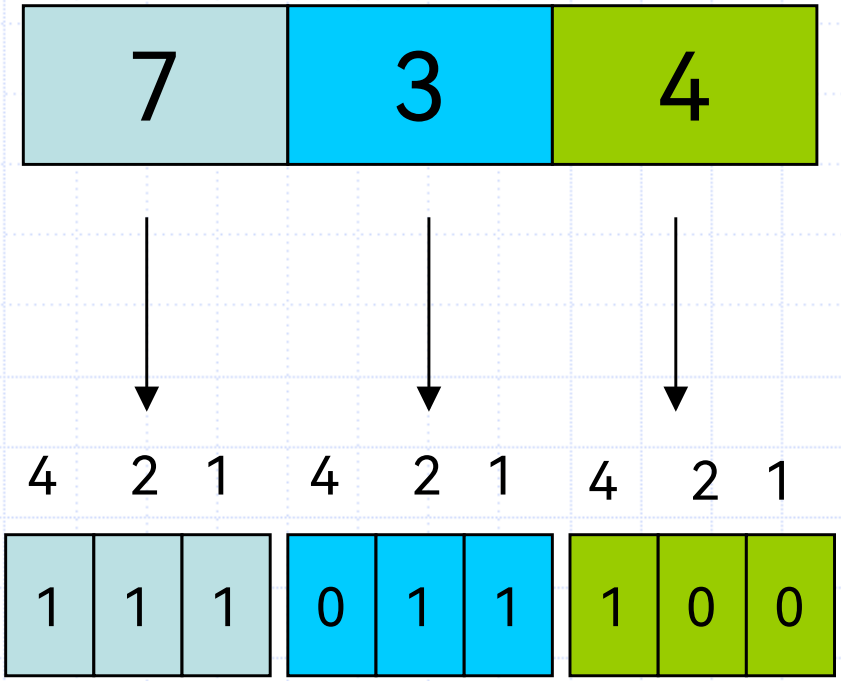
- Hexadecimal به Binaryyy
  - هر رقم را با 4 رقم باینری جایگزین کنید
- $(F25)_{16} = (1111\ 0010\ 0101)_2$



1111 0010 0101

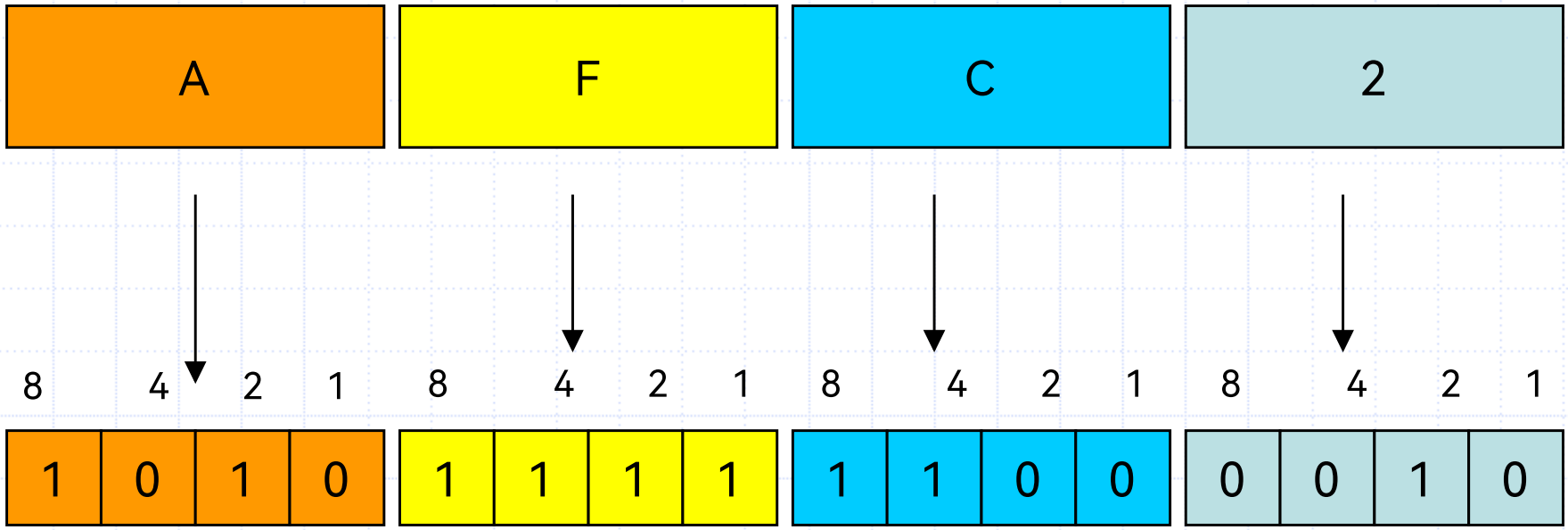
مثال

- عدد $(734)_8$ را به مبنای 2 تبدیل نمایید



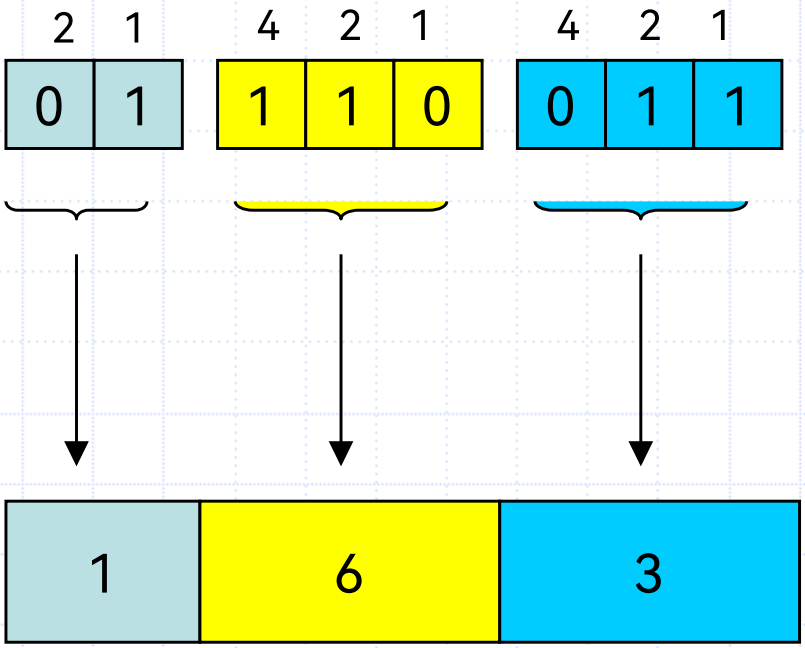
مثال

- عدد $(AFC2)_{16}$ را به مبنای 2 تبدیل نمایید :



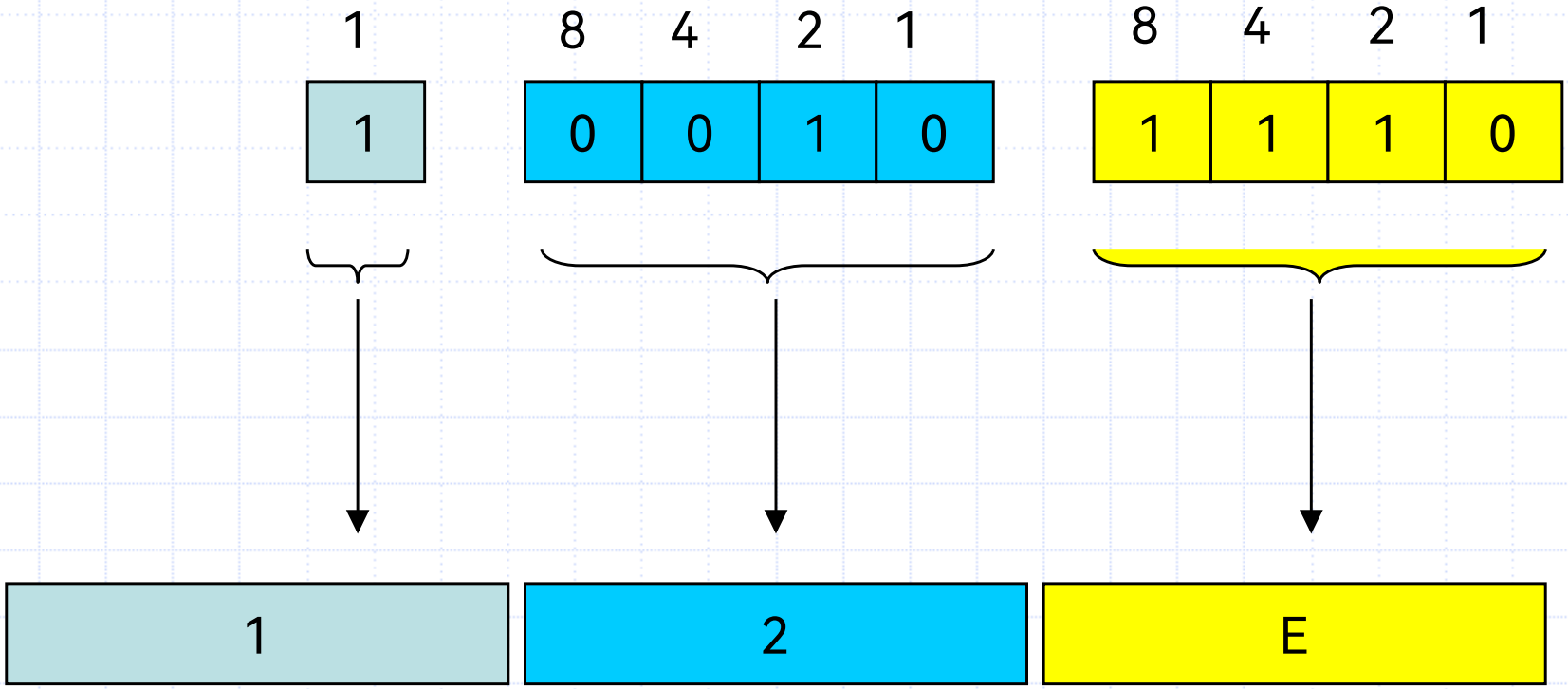
مثال

- عدد $(01110011)_2$ را به مبنای 8 تبدیل کنید



مثال

- عدد $(100101110)_2$ را به مبنای 16 تبدیل کنید :



اعداد علامت دار

در حالت طبیعی سیستم اعداد انتخاب شده برای نمایش اعداد باید بتواند هم اعداد بدون علامت و هم اعدادعلامت دار را نمایش دهد.

سه روش زیر برای نمایش اعداد علامت دار وجود دارد:

نمایش بصورت علامت مقدار: سمت چپ ترین بیت نشان دهنده علامت (0 عدد مثبت و 1 عدد منفی)

نمایش بصورت مکمل یک: از عدد مثبت مکمل یک می گیریم

نمایش بصورت مکمل دو: از عدد مثبت مکمل دو می گیریم

تعداد بیت های در نظر گرفته شده اهمیت دارد

اعداد علامت دار (مثال)

اعداد 9+ و 9- را به صورت یک عدد باینری مبنای ۲ با استفاده از ۳روش اسلاید قبل و با استفاده از ۸ بیت نمایش دهید:

تنها یک روش برای نمایش عدد 9+ وجود دارد:

0	0	0	0	1	0	0	1

برای عدد 9- سه روش نمایش وجود دارد:

روش علامت مقدار:

1	0	0	0	1	0	0	1

روش مکمل یک:

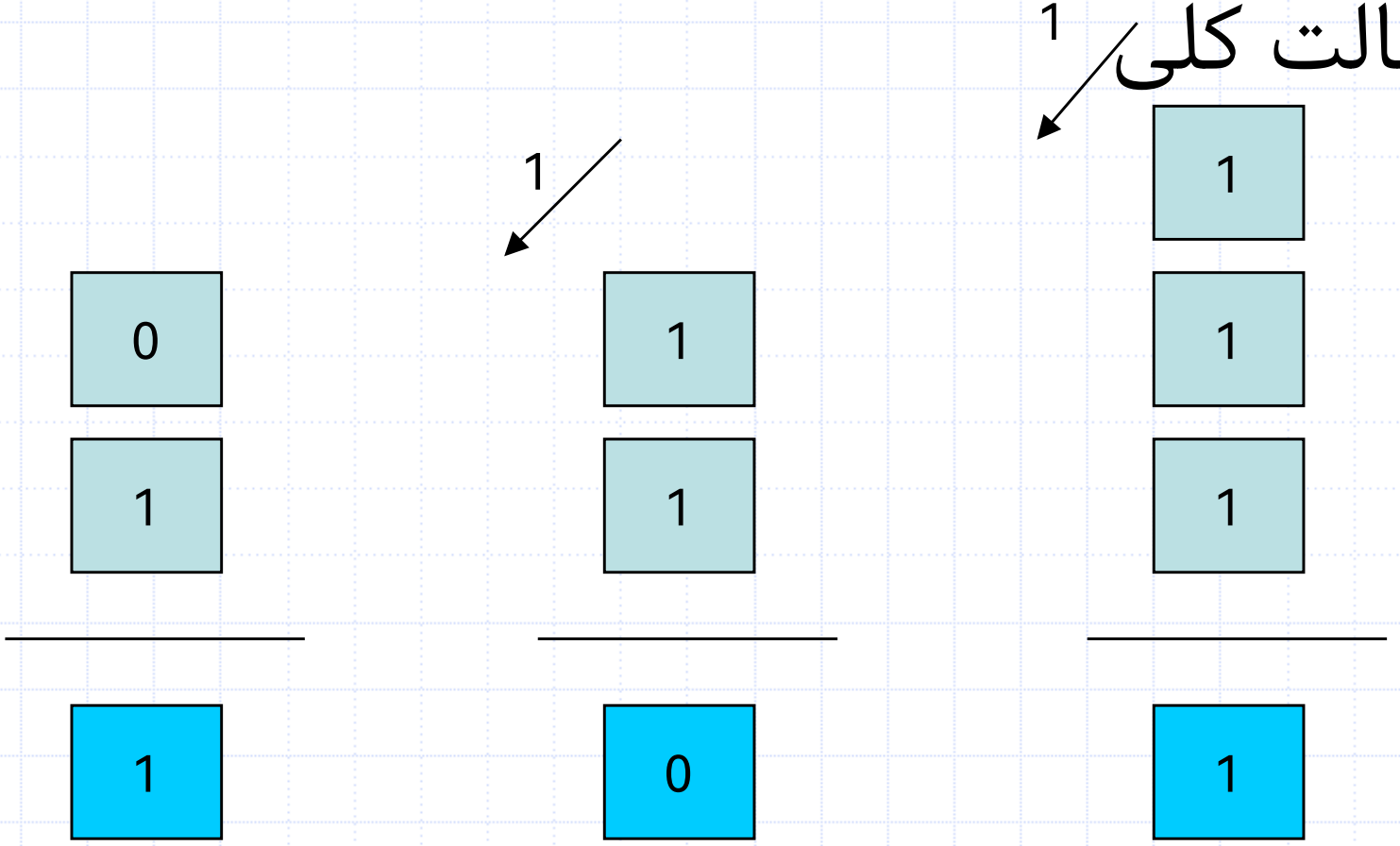
1	1	1	1	0	1	1	0

روش مکمل دو:

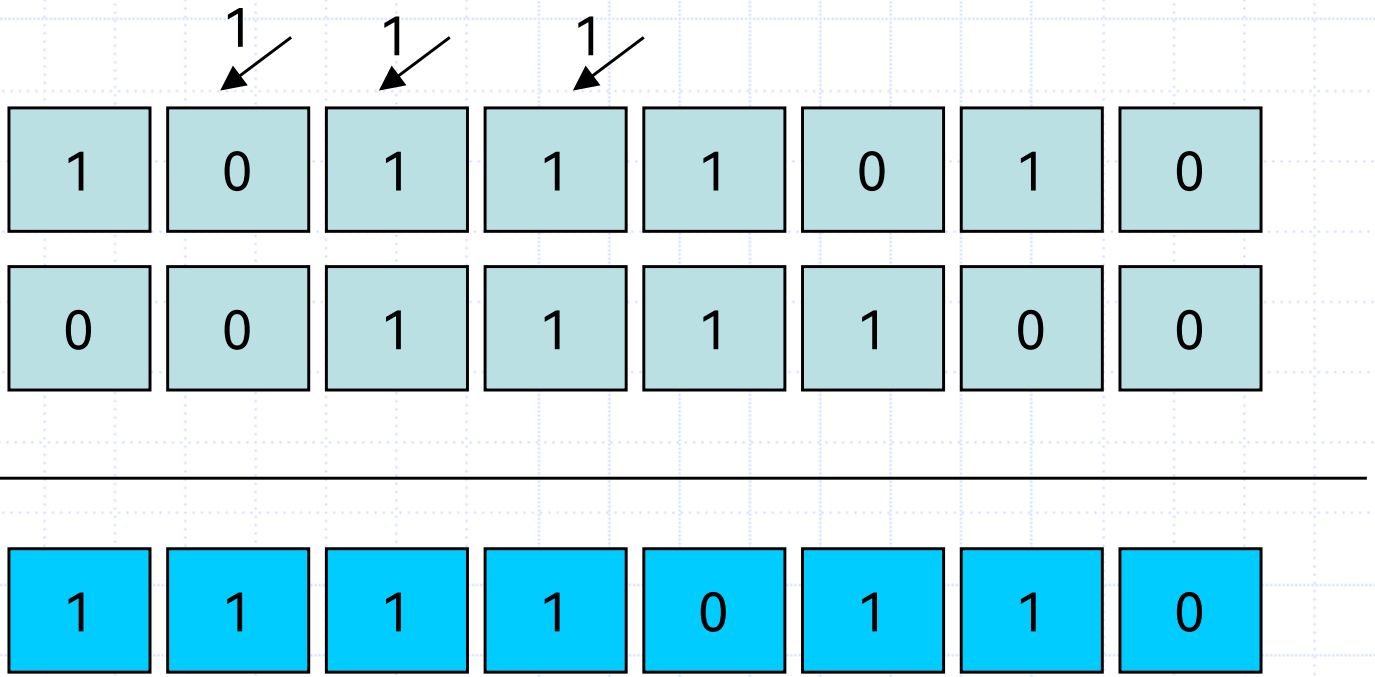
1	1	1	1	0	1	1	1

جمع در مبنای 2

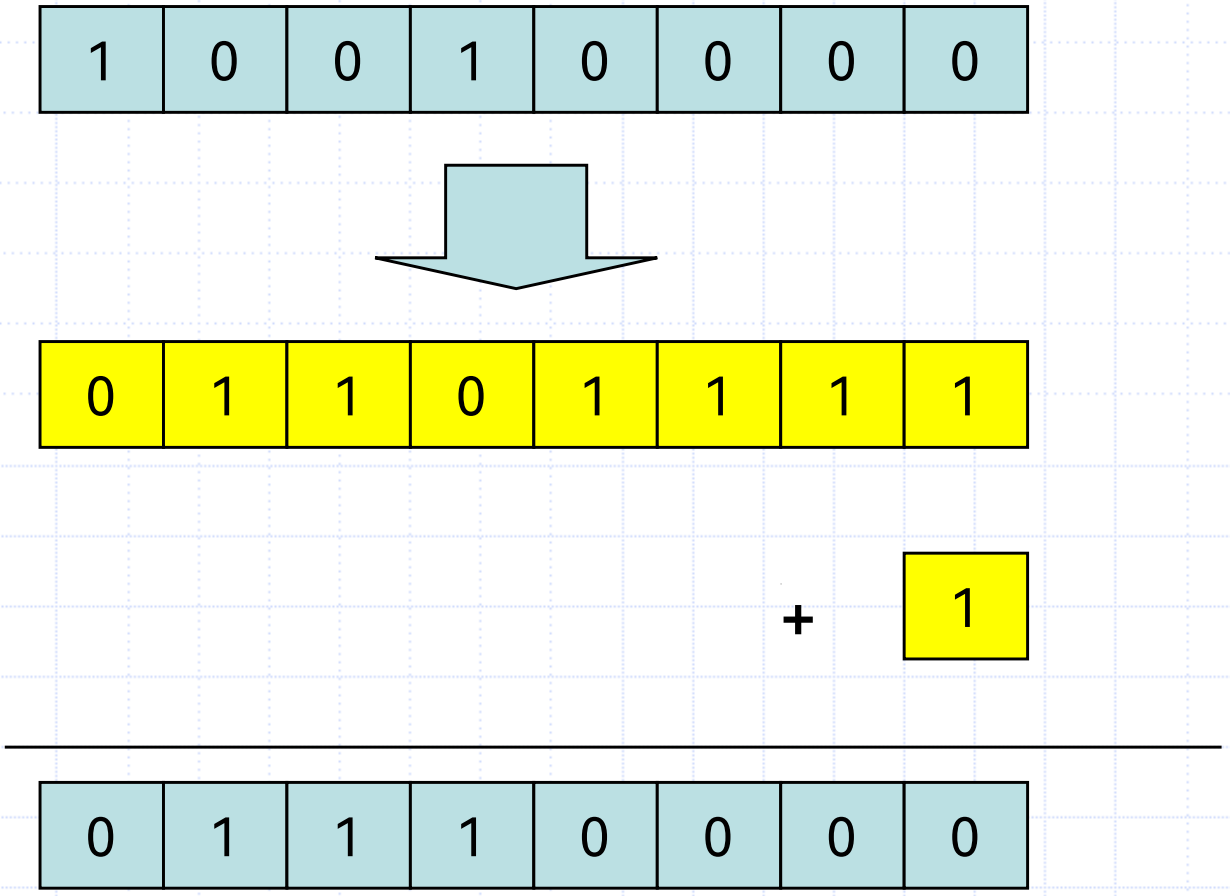
- سه حالت کلی



مثال



مثال



روش سریع بدست آوردن مکمل دو

1. از سمت راست شروع کنید و هر بیت را به خروجی منتقل نمایید تا زمانی که به اولین یک برسید
2. اولین یک را نیز به خروجی منتقل کنید
3. بقیه بیت ها را به صورت نقیض در خروجی بنویسید

مثال :

1	0	0	1	0	0	0	0

0	1	1	1	0	0	0	0

نقیض

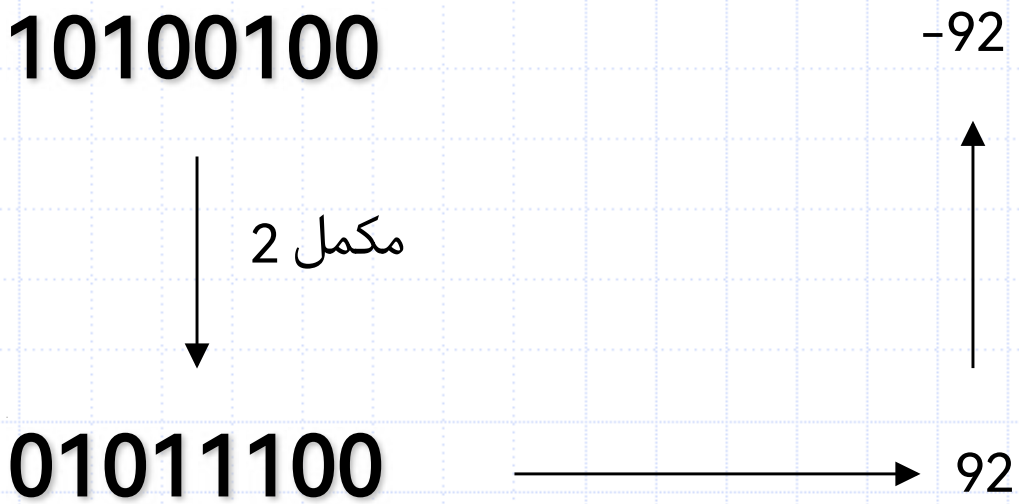
بدون تغییر

تبدیل اعداد باینری منفی به دهدهی

- هیچ روش مستقیمی برای این کار وجود ندارد

- ابتدا : قرینه عدد (مکمل دو) را تولید می کنیم
- مقدار قرینه عدد را در مبنای ده بدست می آوریم
- یک علامت منفی به جواب اضافه می کنیم

مثال : عدد (10100100) در سیستم اعداد علامتدار می باشد، معادل مبنای ده آن چند می باشد؟



تفریق

- پردازنده ها عموما فاقد مدار مستقل جهت تفریق هستند

- برای شبیه سازی تفریق :
- عدد اول را با قرینه عدد دوم جمع می کنیم :

$$A - B = A + \overline{B} + 1$$

مکمل دو

- A-B= A+ (-B)
- در سیتم مکمل دو ، قرینه عدد همان مکمل عدد است

فصل دوم

روش های جبری برای تحلیل و طراحی مدارهای منطقی

دستگاه های دیجیتالی

- جبر بول:
  - یک عبارت منطقی می تواند "درست" یا " نادرست" باشد (0 یا 1).
  - شامل فرمول های جبری مربوط به ترکیب های مقادیر منطقی است.
  - در ریاضیات و به خصوص در منطق ریاضی، جبر بولی زیر مجموعه‌ای از جبر است که در آن مقدار متغیرها، درست یا غلط می‌باشد.
  - به جای جبر مقدماتی که در آن مقدار متغیرها اعداد هستند و عملگرهای اصلی جمع و ضرب می‌باشند، عملگرهای اصلی جبر بولی عطف منطقی و که با ∧ نشان داده می‌شود، فصل منطقی یا که با ∨ نشان داده می‌شود و نقیض که با ¬ نشان داده می‌شود، می‌باشند.

- در سطح سخت افزار:

دستگاه های دیجیتالی(2)

مثال:

سطح ولتاژ بالا ← عبارت درست است.

سطح ولتاژ پائین ← عبارت نادرست است.

- عملگرهای منطقی با گیت های منطقی پیاده سازی می شوند.

اصول جبر بول (1)

اصول اساسی:

اصل 1:

تعریف: برای هر a و b که متعلق به مجموعه ی K هستند، $a.b$ و $a+b$ نیز به مجموعه ی K تعلق دارند.

Or $a+b$ *And* $a.b$

و). ، نامیده می شود (،

$$\text{If } a \,\&\, b \in K \rightarrow \begin{cases} a.b \in K \\ a+b \in K \end{cases}$$

اصول جبر بول (2)

اصل 2:

موجودیت عناصر 0و 1:

$x + 0 = x$

$x \cdot 1 = x$

x	x + 0	x . 1
0	0	0
1	1	1

اصول جبر بول (3)

اصل 3:

$$x + y = y + x$$

$$x \cdot y = y \cdot x$$

خاصیت عناصر + و . :

x	y	x.y	y.x	x+y	y+x
0	0	0	0	0	0
0	1	0	0	1	1
1	0	0	0	1	1
1	1	1	1	1	1

اصول جبر بول (4)

x	y	x+y	x.y	y.x’	x’
0	0	0	0	0	1
0	1	1	0	1	1
1	0	1	0	0	0
1	1	1	1	1	0

اصول جبر بول (5)

4 اصل:

خاصیت شرکت پذیری اعمال + و.

$$(x + y) + z = x + (y + z)$$

$$x . (y . z) = (x . y) . z$$

اصول جبر بول (6)

5 اصل:

+: خاصیت توزیع پذیری + بر . و . بر

$$x .(y + z) = x . y + x . z$$

$$x +(y . z) = (x + y) . (x + z)$$

آزمون درستی توزیع پذیری + بر . و بر + (2)

=

x	y	z	y.z	x+(y.z)	x+y	x+z	(x+y)(x+z)
0	0	0	0	0	0	0	0
0	0	1	0	0	0	1	0
0	1	0	0	0	1	0	0
0	1	1	1	1	1	1	1
1	0	0	0	1	1	1	1
1	0	1	0	1	1	1	1
1	1	0	0	1	1	1	1
1	1	1	1	1	1	1	1

اصول اساسی جبر بول (1)

1.خاصیت خود توانی:

a + a = a

a . a = a

2.و . : + عناصر بی اثر در

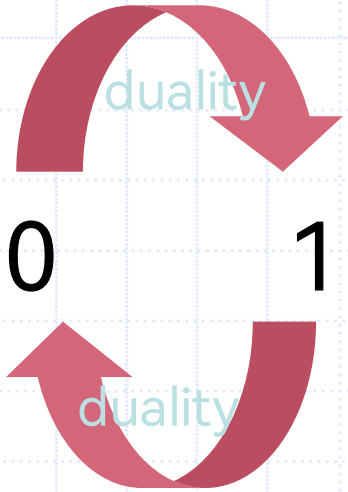
a . 1 = a

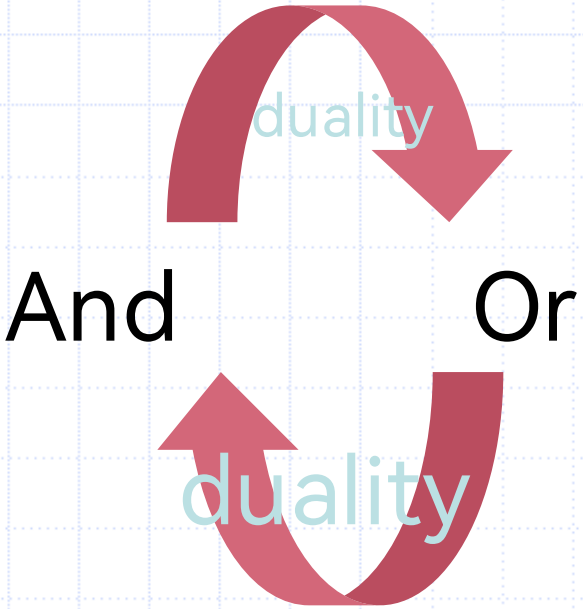
a + 0 = a

3.متمّمِ متمّم:

a’’ = a

دوگان (duality)





مثال:

$x+y'z \longleftrightarrow x.(y'+z)$ دوگان

یک تابع با دوگان آن از نظر ارزش، ارتباطی با هم ندارند.

اگر رابطه ای در جبر بول برقرار باشد، دوگان آن رابطه نیز برقرار است.

قوانین دمرگان(2)

$$\overline{A.B} = \overline{A} + \overline{B}$$

ورودی‌ها		خروجی‌های جدول درستی برای هر عبارت				
A	B	A. B	$\overline{A.B}$	$\overline{A}$	$\overline{B}$	$\overline{A} + \overline{B}$
0	0	0	**1**	1	1	**1**
0	1	0	**1**	1	0	**1**
1	0	0	**1**	0	1	**1**
1	1	1	**0**	0	0	**0**

برای یافتن مکمل یک تابع میتوان ابتدا آن را دوگان کرده و سپس متغیرهایش را مکمل کنیم

- مینترم

- با داشتن n متغیر می توان 2^n جمله به صورت حاصلضرب و متمایز از یکدیگر بدست آورد که در هر جمله، هر متغیر به صورت عادی و یا متمم حضور دارد. این جملات از صفر تا 2^n-1 شماره گذاری می گردند و هر یک را با m نشان می دهند.

- بطور **همزمان** خود متغیر و متمم آن نمی تواند در **ضرب فوق** حضور داشته باشد.

- در مینترم متغیر به صورت عادی ظاهر می شود اگر ارزش متغیر در جدول درستی برابر یک باشد و به صورت مکمل ظاهر می شود هرگاه ارزش متغیر برابر صفر باشد.

مینترم –

x	y	z	Minterm	
0	0	0	$x'.y'.z'$	m_0
0	0	1	$x'.y'.z$	m_1
0	1	0	$x'.y.z'$	m_2
0	1	1	$x'.y.z$	m_3
1	0	0	$x.y'.z'$	m_4
1	0	1	$x.y'.z$	m_5
1	1	0	$x.y.z'$	m_6
1	1	1	$x.y.z$	m_7

ـ نحوه نمایش توابع بولي

- ترکیبي نامرتب از عبارات و متغیرهاي بولي
- مجموع مینترمها(جمع ضربها)

Canonical Form
Standard Form

- Canonical Forms :
  - Sum of minterms (SOP)

- برای هر تابع فقط یک فرم استاندارد SOP وجود دارد.

ـ حاصل جمع مینترمها

- در جدول درستی زیر F1 را به صورت حاصل جمع مینترمها بنویسید.

x	y	z	F_1	F_2
0	0	0	0	1
0	0	1	1	0
0	1	0	0	1
0	1	1	0	1
1	0	0	1	0
1	0	1	1	0
1	1	0	1	0
1	1	1	1	0

$$F_1(x, y, z) = m_1 + m_4 + m_5 + m_6 + m_7 = \sum(1,4,5,6,7)$$

$$= (x' y' z) + (xy' z') + (xy' z) + (xyz') + (xyz)$$

تبدیل توابع بولي به مجموع مینترمها

- با استفاده از قضایاي گفته شده در جبر بول، (a+a′=1) در هر جمله عبارت بولي، **تمام** متغیرها را وارد مي‌کنیم.

$F(A,B,C) = AB + AC' + A'C$

$$F(A,B,C) = AB.1 + AC'.1 + A'C.1$$

$$= AB.(C + C') + AC'.(B + B') + A'C.(B + B')$$

$$= ABC + ABC' + ABC' + AB'C' + A'BC + A'B'C$$

$$= ABC + ABC' + AB'C' + A'BC + A'B'C$$

$$= \Sigma(1,3,4,6,7)$$

نمایش توابع با مینترم

x	y	z	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

$$F = xyz + xyz' + x'yz$$

$$\downarrow$$

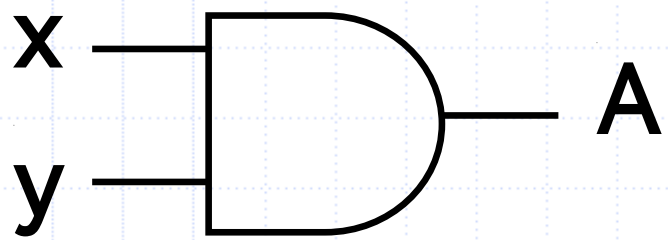
$$F = m_7 + m_6 + m_3 = \Sigma(3, 6, 7)$$

تعبیر دیگری از رفتار گیتی -

- گیت Or، ماکزیمم ورودیها را به خروجی می فرستد.
- گیت And، مینیمم ورودیها را به خروجی می فرستد.
- گیت Nor، مکمل ماکزیمم ورودیها را به خروجی می فرستد.
- گیت Nand، مکمل مینیمم ورودیها را به خروجی می فرستد.
- گیت Xor، تعداد فرد یک را به خروجی می فرستد.
- گیت Xnor، تعداد زوج یک را به خروجی می

گیت ها(دریچه ها) (1)

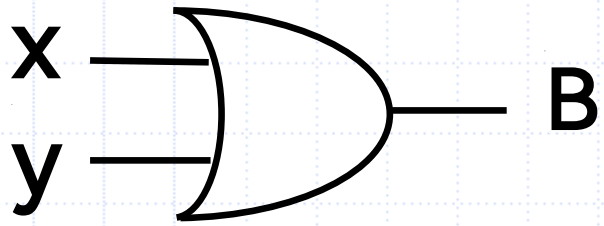
- And:



x	y	A = x . y
0	0	0
0	1	0
1	0	0
1	1	1

گیت ها(دریچه ها) (1)

- Or:

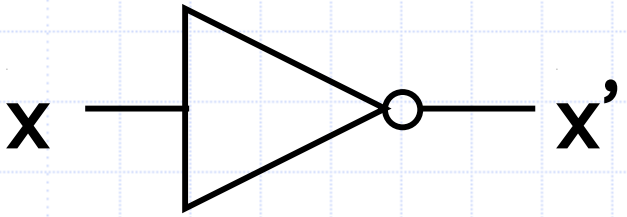


x	y	A = x + y
0	0	0
0	1	1
1	0	1
1	1	1

گیت ها (2)

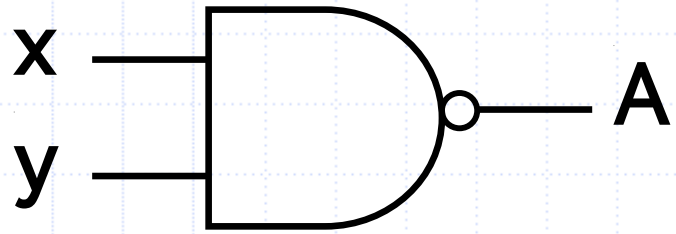
x	x’
0	1
1	0

متمّم:



گیت ها (3)

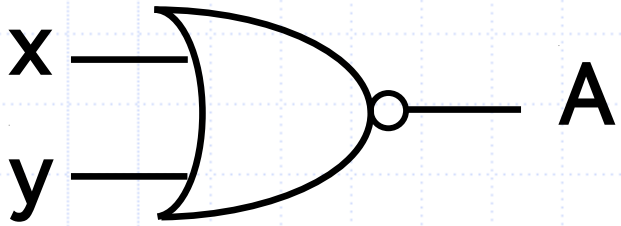
- Nand:



x	y	A
0	0	1
0	1	1
1	0	1
1	1	0

گیت ها (3)

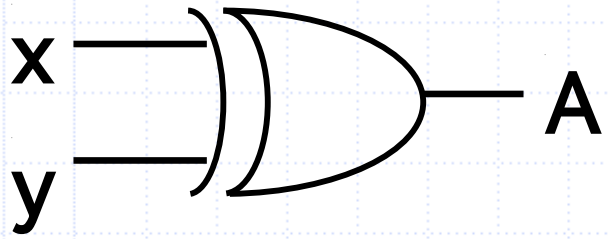
- Nor:



x	y	A
0	0	1
0	1	0
1	0	0
1	1	0

گیت ها (4)

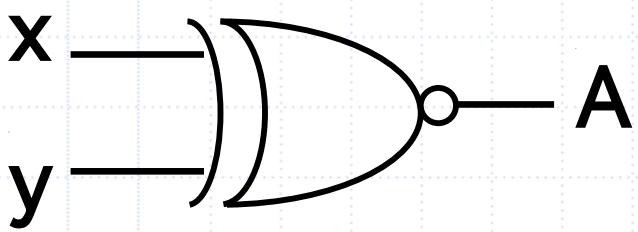
- Xor:



x	y	A
0	0	0
0	1	1
1	0	1
1	1	0

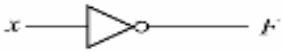
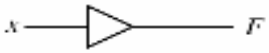
گیت ها (4)

- Xnor:



x	y	A
0	0	1
0	1	0
1	0	0
1	1	1

سمبولهای استاندارد گیتی -

Name	Graphic symbol	Algebraic function	Truth table
AND		$F \quad xy$	$x\ y \mid F$: 0 0 | 0; 0 1 | 0; 1 0 | 0; 1 1 | 1
OR		$F = x + y$	$x\ y \mid F$: 0 0 | 0; 0 1 | 1; 1 0 | 1; 1 1 | 1
Inverter		$F = x'$	$x \mid F$: 0 | 1; 1 | 0
Buffer		$F = x$	$x \mid F$: 0 | 0; 1 | 1
NAND		$F = (xy)'$	$x\ y \mid F$: 0 0 | 1; 0 1 | 1; 1 0 | 1; 1 1 | 0
NOR		$F = (x + y)'$	$x\ y \mid F$: 0 0 | 1; 0 1 | 0; 1 0 | 0; 1 1 | 0
Exclusive-OR (XOR)		$F = xy' + x'y$ $= x \oplus y$	$x\ y \mid F$: 0 0 | 0; 0 1 | 1; 1 0 | 1; 1 1 | 0
Exclusive-NOR or equivalence		$F = xy + x'y'$ $(x \oplus y)'$	$x\ y \mid F$: 0 0 | 1; 0 1 | 0; 1 0 | 0; 1 1 | 1

- جدول درستي
- سمبل **استاندارد** گيت
- عمليات منطقی
- Single input يا Multiple

کد گِرِی(1)

در این کد، هر کدام از کد ها تنها در یک بیت با کد قبلی متفاوت است و این روند چرخشی است؛ یعنی آخرین کد و اولین کد نیز تنها در 1 بیت متفاوتند.

کد گری(2)

x	y	z
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

Gray code

BCD code

x	y	z
0	0	0
0	0	1
0	1	1
0	1	0
1	1	0
1	1	1
1	0	1
1	0	0

فصل سوم

ساده سازی توابع بولی

- جدول کارنو (Karnaugh map)

- يكي از بهترين ابزارها براي ساده‌سازي توابع، جدول كارنو است. اين روش اولين بار توسط ويچ پيشنهاد شد و سپس توسط كارنو اصلاح گرديد.(دياگرام ويچ)

- براي ساده سازي **توابع با** حداكثر پنج ورودي، مي توان از جدول كارنا استفاده كرد.

- جدول كارنا را مي توان به عنوان معادل گرافيكي جدول درستي در نظر گرفت.

- هر **حالت از جدو**ل درستي معادل با يک **مربع(خانه)** از نقشه كارنو است. بنابراين تعداد خانه هاي جدول كارنو، با توجه به تعداد ورودي ها بدست مي آيد.

مشخصات جدول کارنو

- هر تابع بولي را مي‌توان بصورت مجموعي از مينترمها نشان داد. جدول کارنو از مربع هايي تشكيل شده است كه **هر مربع، نشان دهنده يک مينترم است.**
- سطرها و **ستونهاي** اين جدول به روش **كد گري**، كدگذاري مي گردند.

هر دومربع همسايه فقط در يک متغير با هم اختلاف دارند. لذا مي توان با توجه به خواص جبر بول، مجموع آنها را ساده كرد.(متغير مشترک را حذف نمود)

خانه‌هايي از جدول كه مقدارتابع در آنها برابر با يك مي‌باشد را مشخص مي‌كنيم.

- بطور كلي براي توابع با n **متغير**، جدول كارنو داراي 2^n

الگوريتم ساده سازي توابع با جدول كارنا ـ

انتقال تابع يا جدول درستي به نقشه كارنو

2. پيدا كردن يكهايي در داخل نقشه كه با هيچكدام از يكهاي داخل نقشه همسايه نباشد. اين يكها به صورت مجزا در حلقه قرار مي‌گيرند و شامل يک عبارت مينترم است.

3. **يافتن بزرگترين حلقه از يكها، كه حداقل بيش از يک ‘1’ آن قبلا داخل حلقه‌اي قرار گرفته نباشد. اين مرحله تازمانيكه همه يكها جدول پوشش(Cover) داده نشده باشند، تكرار مي‌گردد.**

1. در پايان جمله نمايشگر هر حلقه با حلقه هاي ديگر Or مي‌گردد.

نكته يک: تعداد حلقه هاي پيدا شده بايد مينيمم **باشد.**
نكته دو: امكان Overlap شدن حلقه ها وجود **دارد.**

مثال از جدول کارناي دو متغيره –

m_0	m_1
m_2	m_3

x \ y	0	1
0	$x'y'$	$x'y$
1	xy'	xy

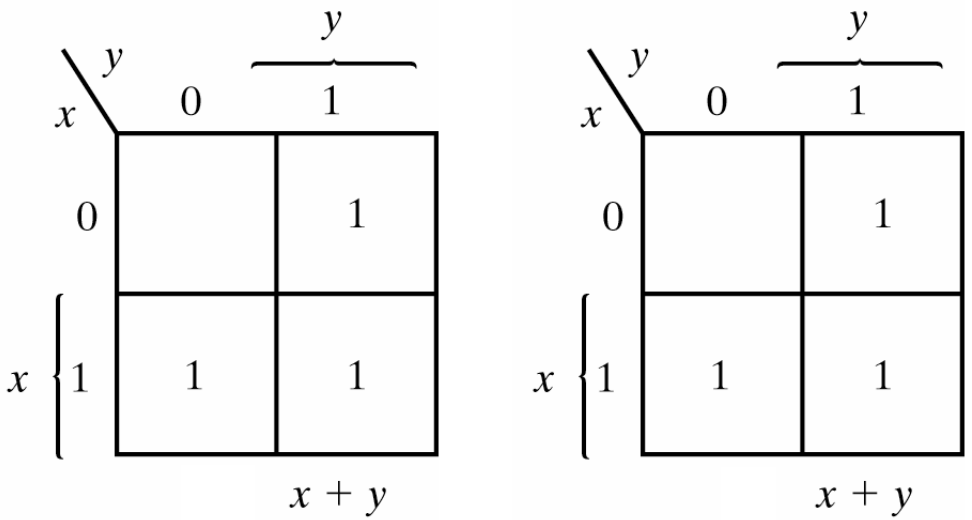
مثال از جدول كارناي دو متغيره -

توابع زیر را توسط جدول کارنو نمایش دهید.

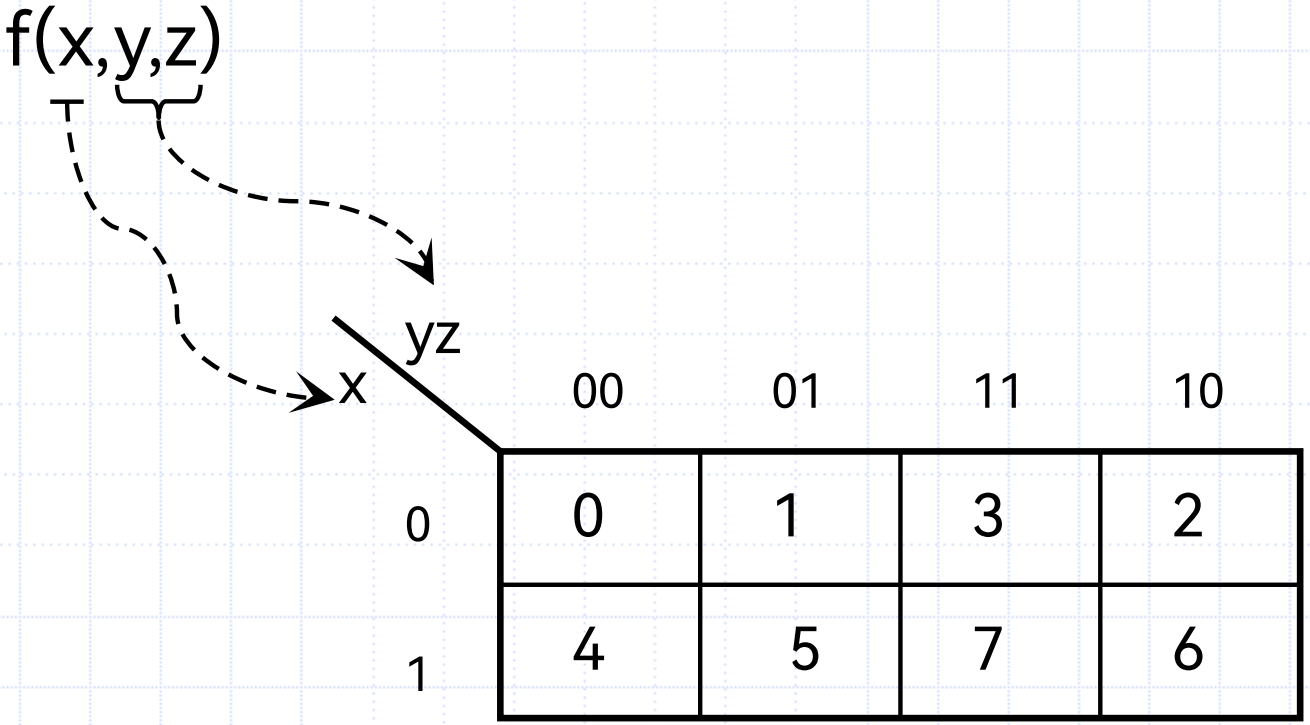
$$F_1(x,y) = xy' + xy + x'y$$

$$F_2(x,y) = x + y$$

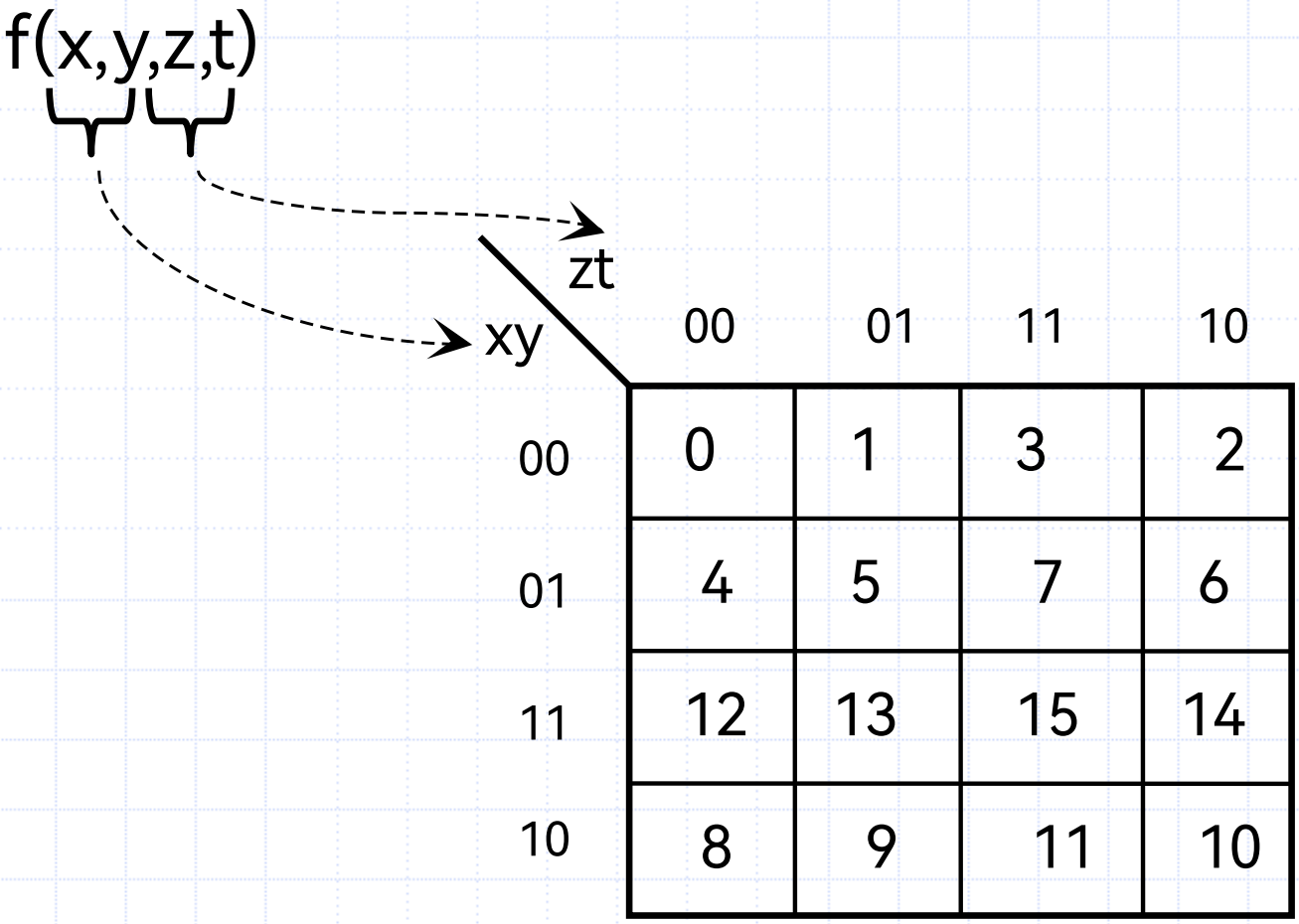
- همانگونه که دیده می‌شود، هر دو تابع به یك صورت در جدول ظاهر شده اند. همچنین با استفاده از قوانین جبر بول مي توان f_1 را ساده کرد و به f_2 رسید.



جدول کارنا برای 3 ورودی



جدول کارنا برای 4 ورودی



مثال- جدول کارناي سه متغيره

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6

x \ yz	00	01	11	10
0	$x'y'z'$	$x'y'z$	$x'yz$	$x'yz'$
1	$xy'z'$	$xy'z$	xyz	xyz'

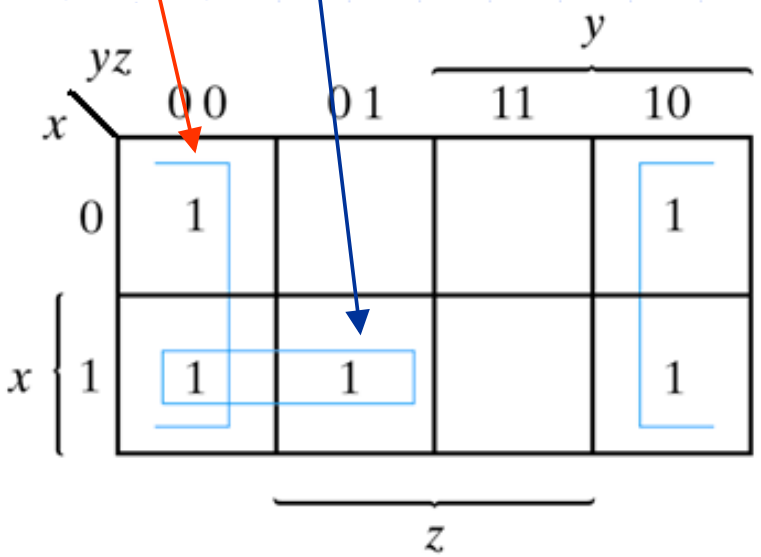
y (11, 10)

z (01, 11)

مثال جدول کارناي سه - متغيره

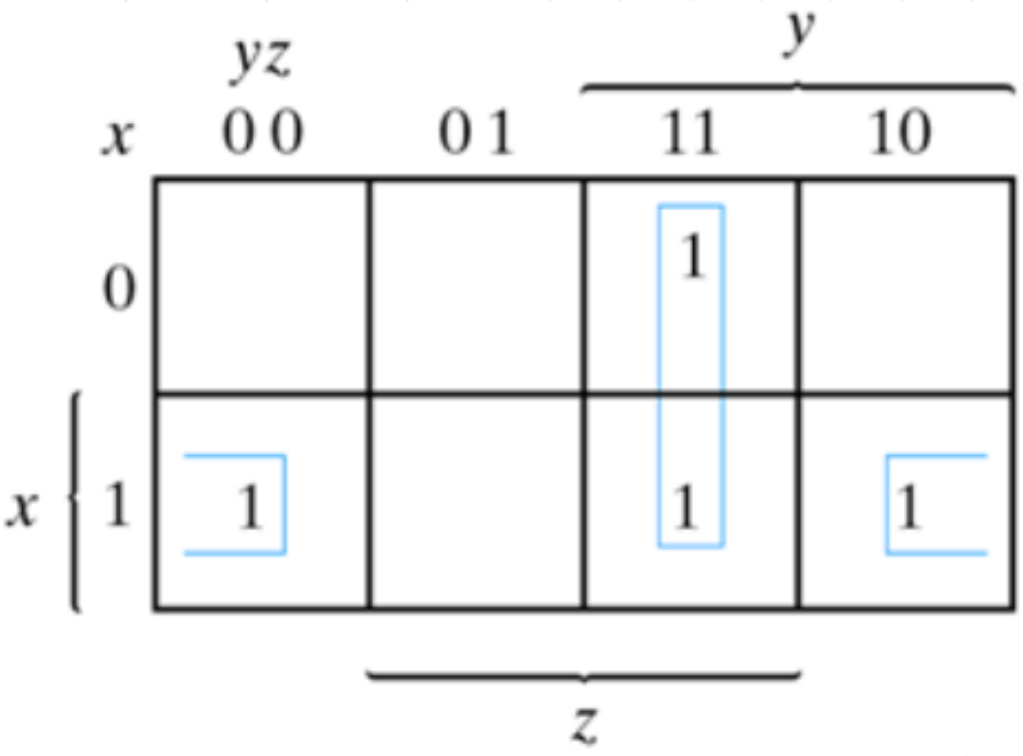
$F(x, y, z) = \sum(0,2,4,5,6)$

$F = z' + xy'$



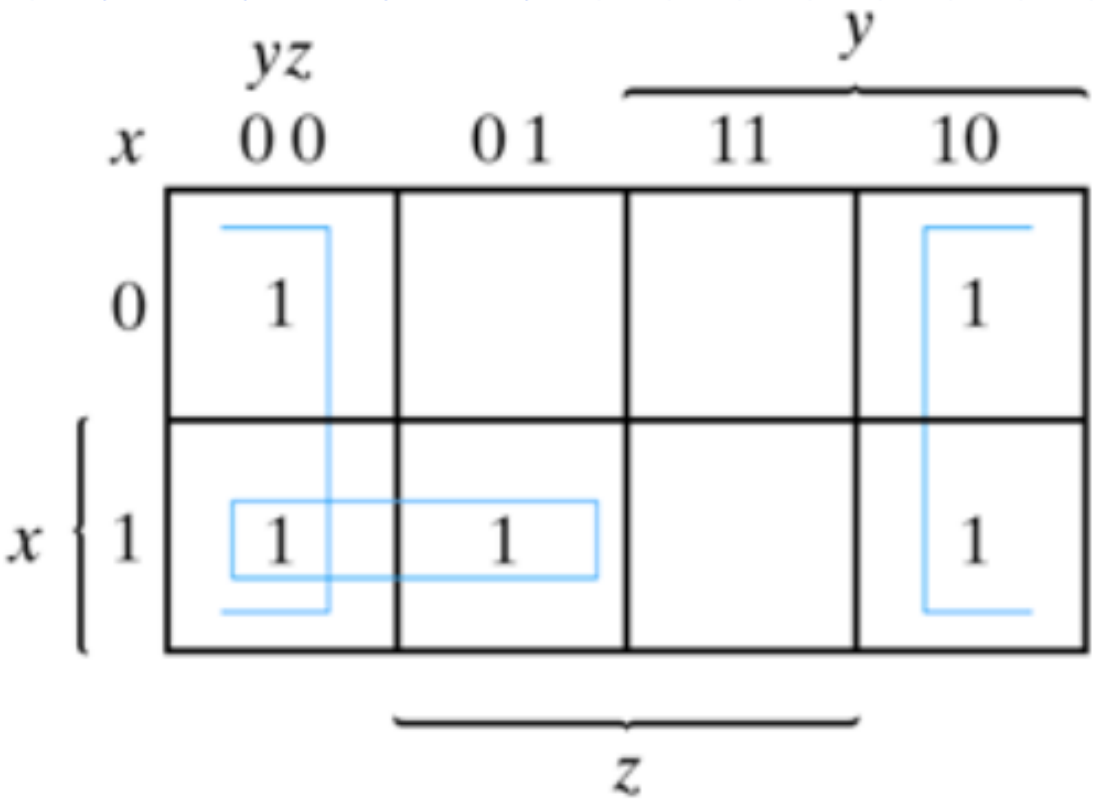
- یک مربع نشان دهنده یک مینترم است ← یک ترم سه متغیره
- دو مربع همسایه ← یک ترم دو متغیره
- چهار مربع همسایه ← یک ترم یک متغیره
- هشت مربع همسایه ← تابع همیشه یک

مثال جدول کارناي سه - متغيره



$$F(x, y, z) = \Sigma(3, 4, 6, 7) = yz + xz'$$

مثال- جدول کارناي سه متغيره

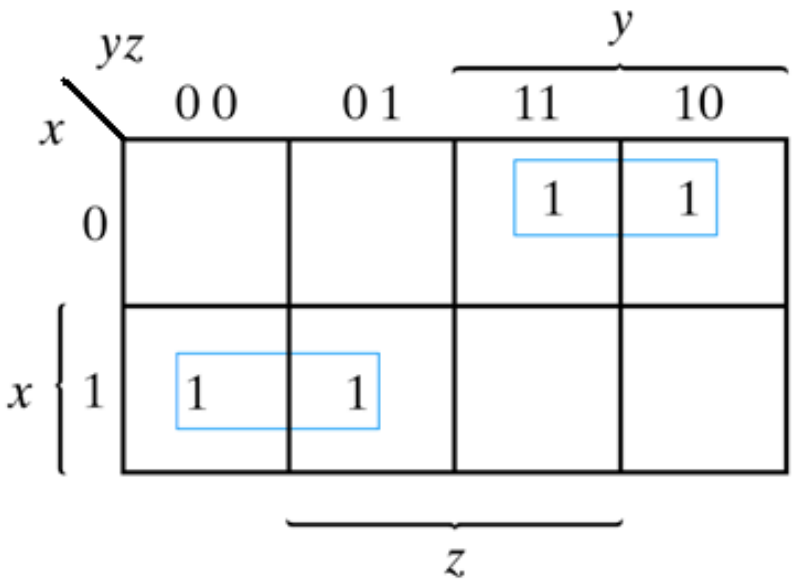


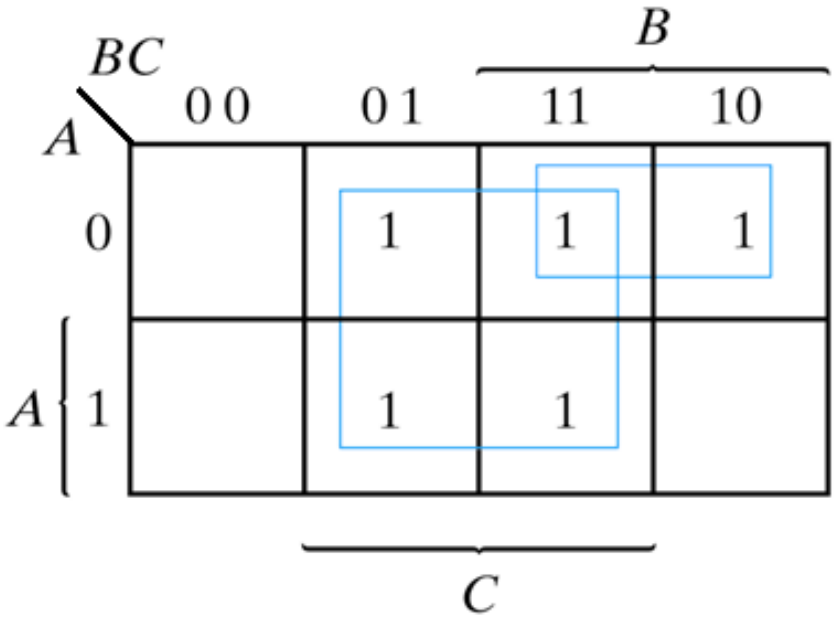
$$F(x, y, z) = \Sigma(0, 2, 4, 5, 6) = z' + xy'$$

مثال هایي دیگر -

$F(x, y, z) = \Sigma(2,3,4,5)$

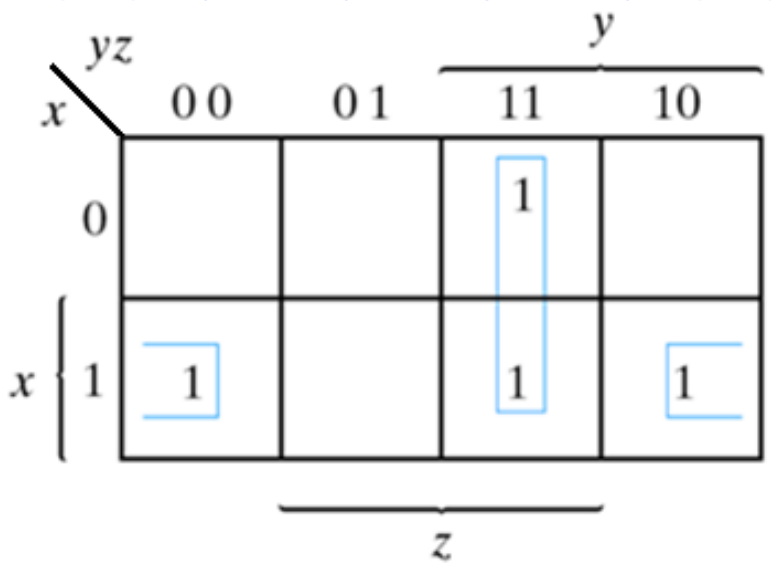
$F = xy' + x'y$





$F(A, B, C) = \Sigma(1,2,3,5,7)$

$F = C + A'B$



$F(x, y, z) = \Sigma(3,4,6,7)$

$F = yz + xz'$

در جدول کارناي سه متغيره -

- یک مربع نشان دهنده یک مینترم است ← یک ترم سه متغیره
- دو مربع همسایه ← یک ترم دو متغیره
- چهار مربع همسایه ← یک ترم یک متغیره
- هشت مربع همسایه ← تابع همیشه یک

در جدول کارناي چهار متغيره -

- یک مربع نشان دهنده یک مینترم است ← یک ترم چهار متغیره
- دو مربع همسایه ← یک ترم سه متغیره
- چهار مربع همسایه ← یک ترم دو متغیره
- هشت مربع همسایه ← یک ترم یک متغیره
- شانزده مربع همسایه ← تابع همیشه یک

مثال از جدول کارناي چهار متغيره -

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6
m_{12}	m_{13}	m_{15}	m_{14}
m_8	m_9	m_{11}	m_{10}

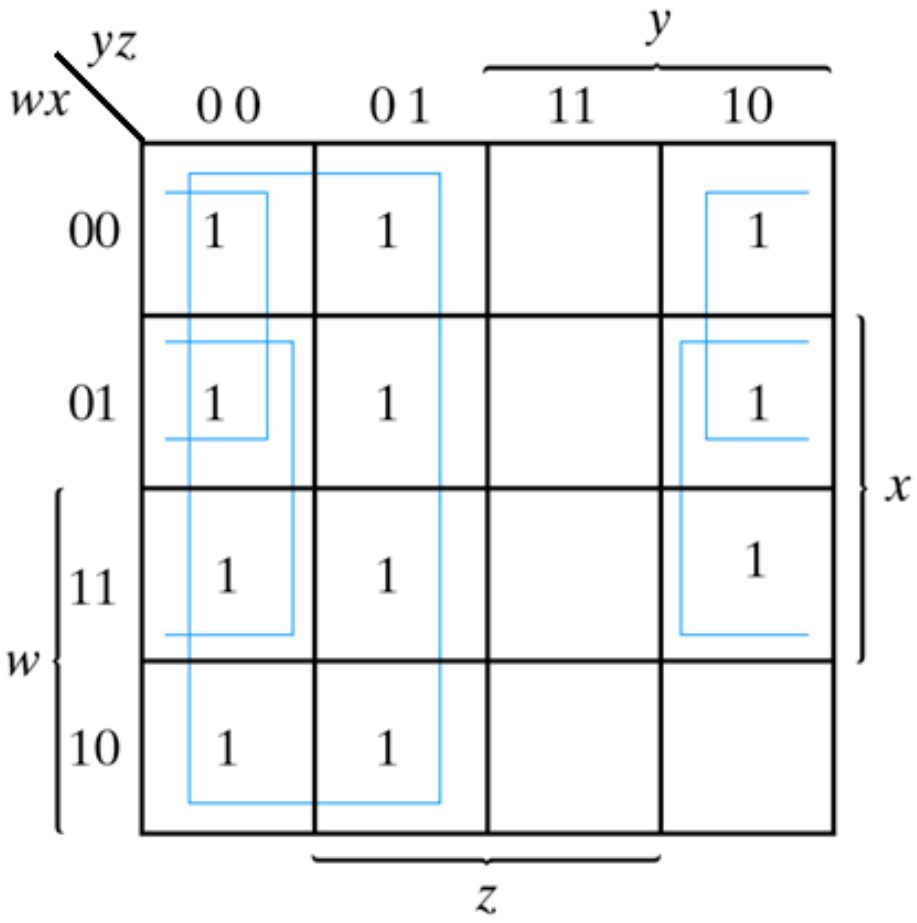
wx \ yz	00	01	11	10
00	$w'x'y'z'$	$w'x'y'z$	$w'x'yz$	$w'x'yz'$
01	$w'xy'z'$	$w'xy'z$	$w'xyz$	$w'xyz'$
11	$wxy'z'$	$wxy'z$	$wxyz$	$wxyz'$
10	$wx'y'z'$	$wx'y'z$	$wx'yz$	$wx'yz'$

y (11, 10) — x — w (11, 10) — z (00, 01)

مثال از جدول کارناي چهار متغيره -

$F(w,x,y,z) = \Sigma(0,1,2,4,5,6,8,9,12,13,14)$

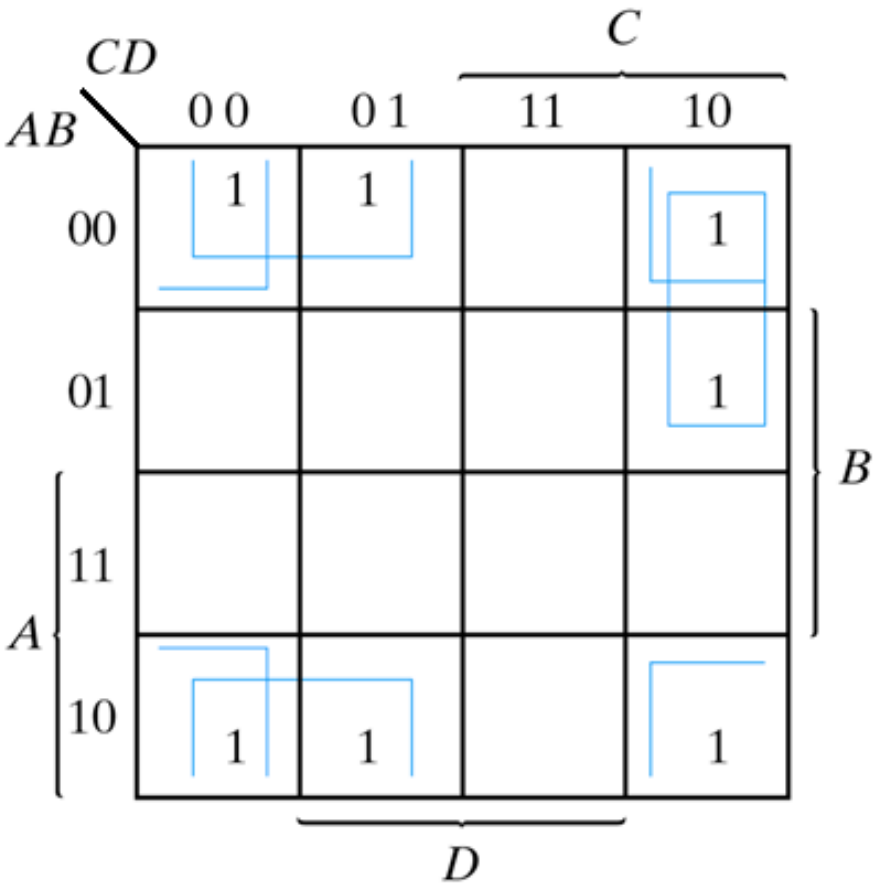
$F = y' + w' z' + xz'$



مثال از جدول کارناي چهار متغيره -

$$F(A,B,C,D) = A'B'C' + B'CD' + A'BCD' + AB'C'$$

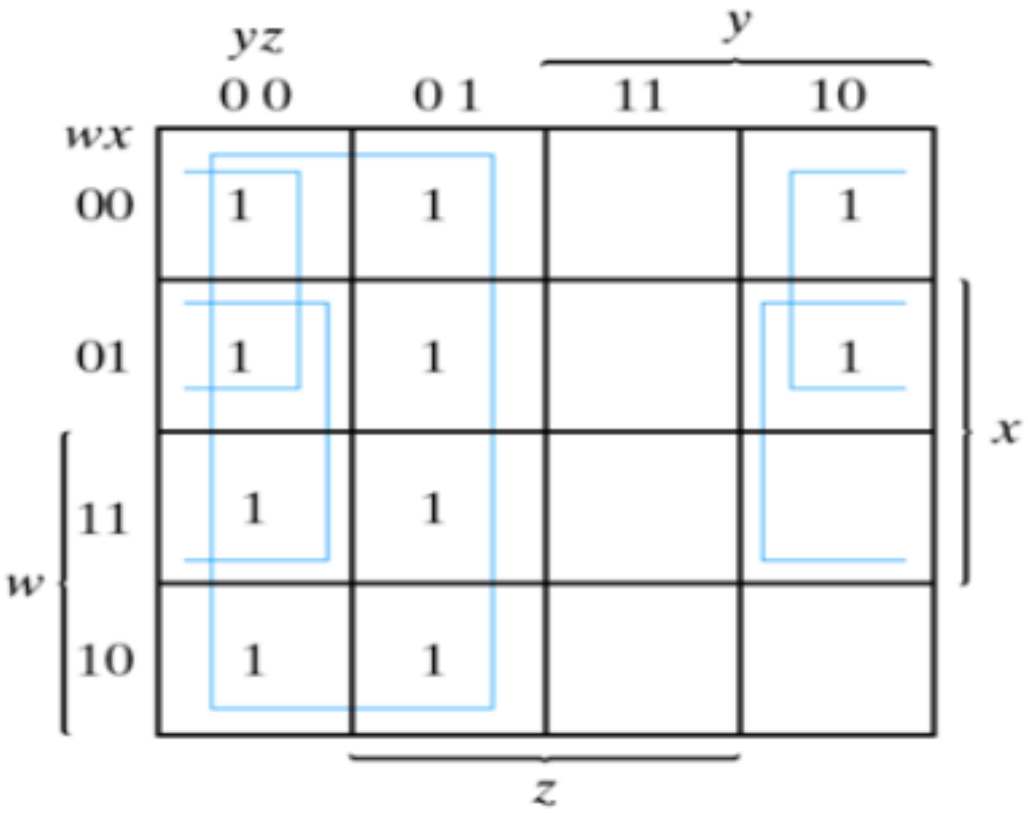
$$F = B'D' + B'C' + A'CD'$$



مثال از جدول کارناي چهار متغيره -

- تابع زير را ساده کنيد:

$$F(w, x, y, z) = \Sigma\,(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$$



$$F(w, x, y, z) =$$
$$= \Sigma\,(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$$
$$= y' + w'z' + xz'$$

دانشگاه صنعتی سجاد

فصل چهارم

مدارهاي منطقي ترکيبي

مدارهاي منطقي -

LOGIC CIRCUITS
1. Combinational
2. Sequential

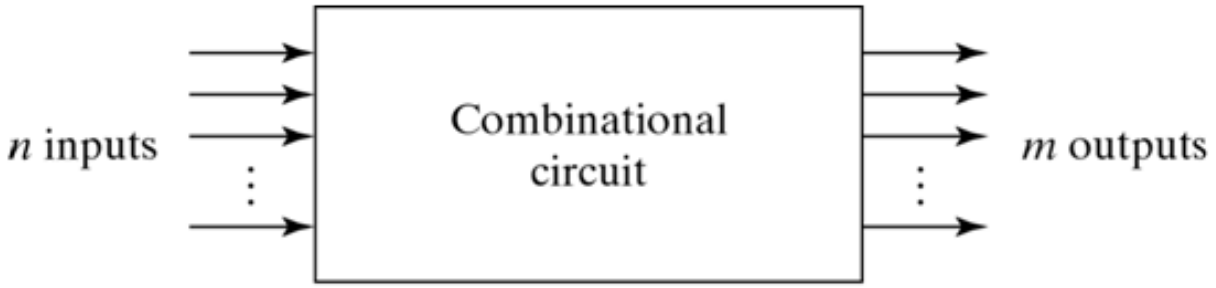
مدارهاي تركيبي: شامل گيتهاي **منطقي است و خروجيهاي آن** در هر زمان مستقيماً از روي **تركيب فعلي ورودیها** بدون توجه به ورودیهاي قبلي تعيين **مي شوند.**
(circuits without a memoryy)

مدارهاي ترتيبي: علاوه بر گيتهاي **منطقي از عناصر حافظه** نيز در آنها استفاده مي‌شود. خروجيهاي آن **تابعي از** عناصر حافظه و ورودي‌هاي مدار هستند. همچنين حالت *عناصر حافظه* نيز تابعي از ورودي‌هاي قبلي است. *(with a memoryy circuits)*

ـ بلوک مدار های ترکیبی

- گروهی از دروازه های منطقی که تعدادی ورودی و تعدادی خروجی دارند.
- مقدار خروجی ها با توجه به محاسباتی که روی مقادیر ورودی انجام می گیرد، تعیین می گردد.

- فرم کلي مدارهاي تركيبي بصورت زير است. مدار مي‌تواند n ورودي و m خروجي داشته باشد.



Half Adder و Full Adder-

- Full Adder (تمام جمع کننده) : يك مدار تركيبي با سه ورودي و دو خروجي است كه دو بيت داده و يك رقم نقلي را با هم جمع كرده و حاصل جمع و رقم نقلي را محاسبه مي كند.

- Half Adder (نيم جمع كننده) : يك مدار تركيبي با دو ورودي و دو خروجي است كه دو بيت دودويي را با هم جمع كرده و حاصل جمع و رقم نقلي را محاسبه مي كند.

ـ روش جمع کردن دو عدد بصورت دستي

قواعد جمع اعداد دودوئي به صورت زير است:

- 0+0=0 0+1=1 1+0=1 1+1=0
- دو عدد دودوئي همانند اعداد دهدهي از راست به چپ رقم به رقم جمع مي شوند.

The initial carry in is implicitly 0

1	1	1	0		رقم نقلي ورودي
		1	0	1	1 جمع شونده
+		1	1	1	0 جمع کننده
	1	1	0	0	1 حاصل جمع

با ارزش ترين بيت

کم ارزش ترين بيت

جمع دو بیت

- براي ساخت يك تمام جمع كننده ابتدا بايد يك نيم جمع كننده كه فقط 2 بيت را جمع مي كند ساخته شود. در اين نيم جمع كننده دو بيت حاصلجمع وجود دارد كه يكي Sum و ديگري Carryyout رقم نقلي خروجي ناميده مي شود.

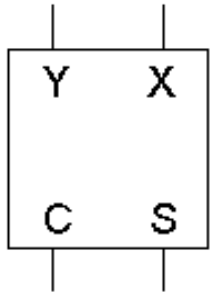
- در زير جدول صحت، دياگرام منطقي و روابط منطقي ميان متغيرهاي ورودي/خروجي نمايش

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

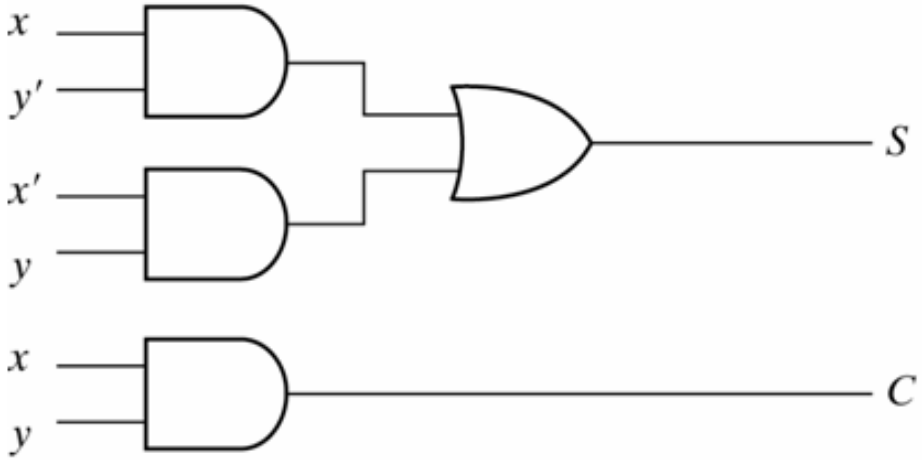
$S=x'y+xy'$

$C=xy$

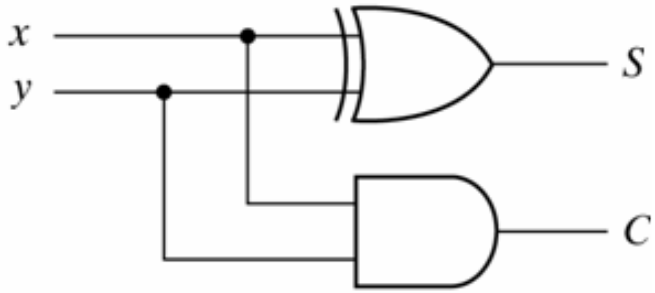
طراحي نيم‌جمع كننده (H.A)



- نيم‌جمع كننده مداري است كه دو بيت ورودي و دو خروج
- دو صورت متفاوت پياده‌سازي مدار نيم‌جمع كننده



$S = xy' + x'y$
$C = xy$



$S = x \oplus y$
$C = xy$

طراحي تمام‌جمع‌كننده

اما چيزي كه واقعاً لازم است جمع سه بيت است.

- يك بيت آن مربوط به جمع كننده، يك بيت جمع شونده و يك بيت رقم نقلي ورودي است.
- تمام‌جمع كننده مداري است كه عمل جمع را بين دو بيت انجام مي‌دهد با اين احتمال كه ممكن است يك بيت نقلي از مرحلة قبل باشد. بنابراين مدار سه ورودي و دو خروجي دارد.
- جدول درستي و توابع بولي آن بصورت زير است:

X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

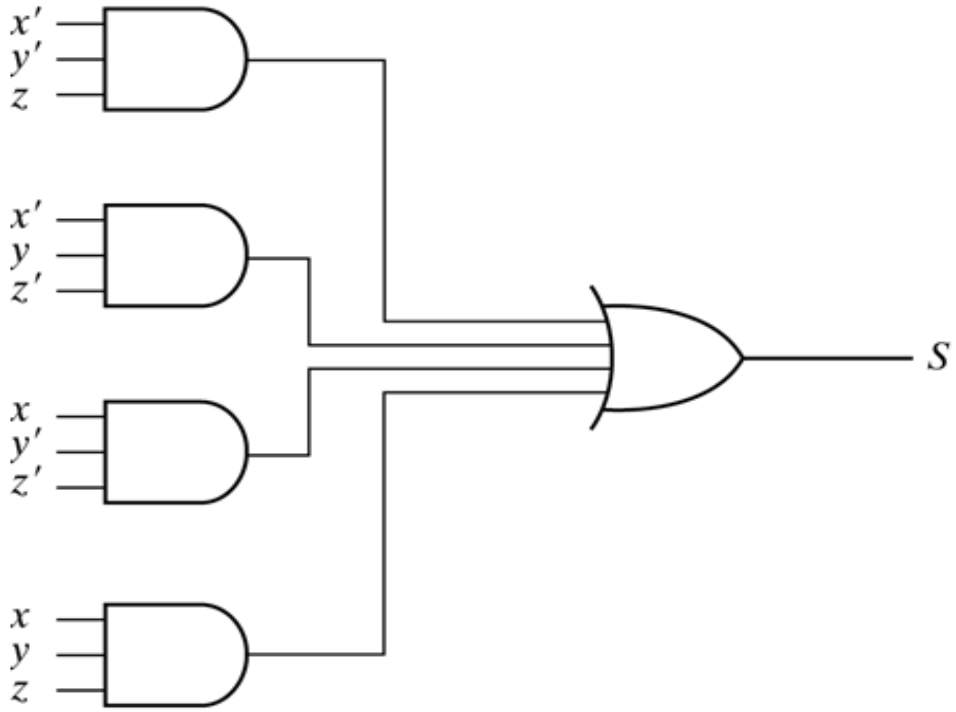
$$S=x'y'z+ x'yz'+xy'z'+xyz=x\oplus y\oplus z$$

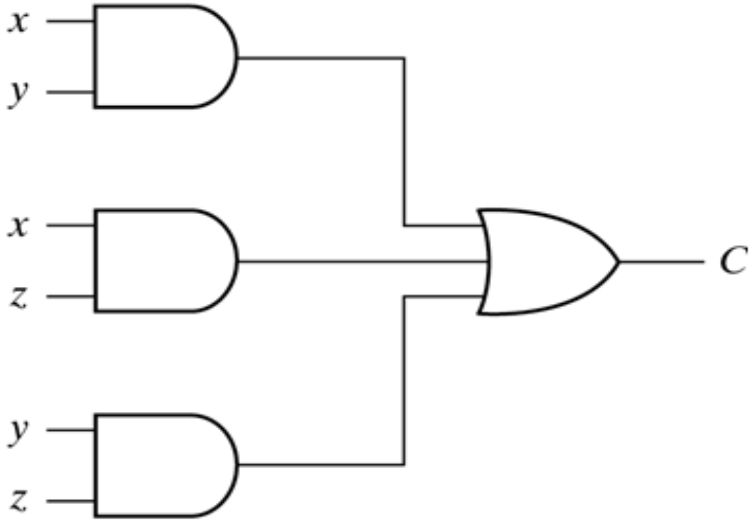
$$C=xy+xz+yz$$

جمع کننده کامل به فرم SOP -

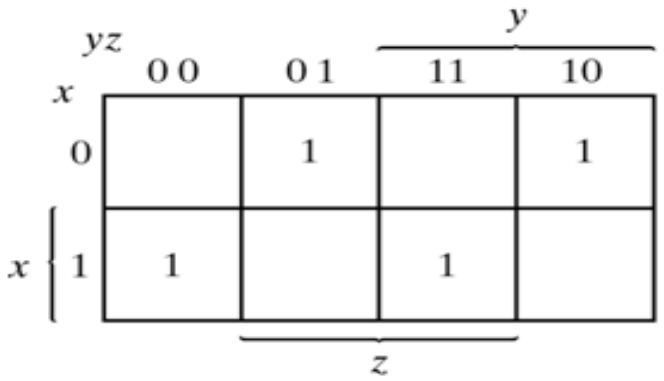
$S=x'y'z+ x'yz'+xy'z'+xyz$

$C=xy+xz+yz$

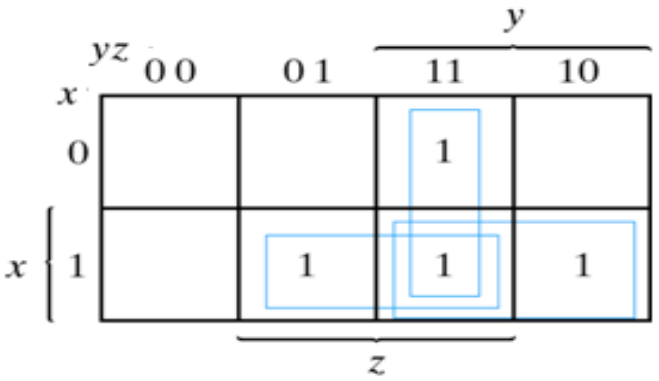




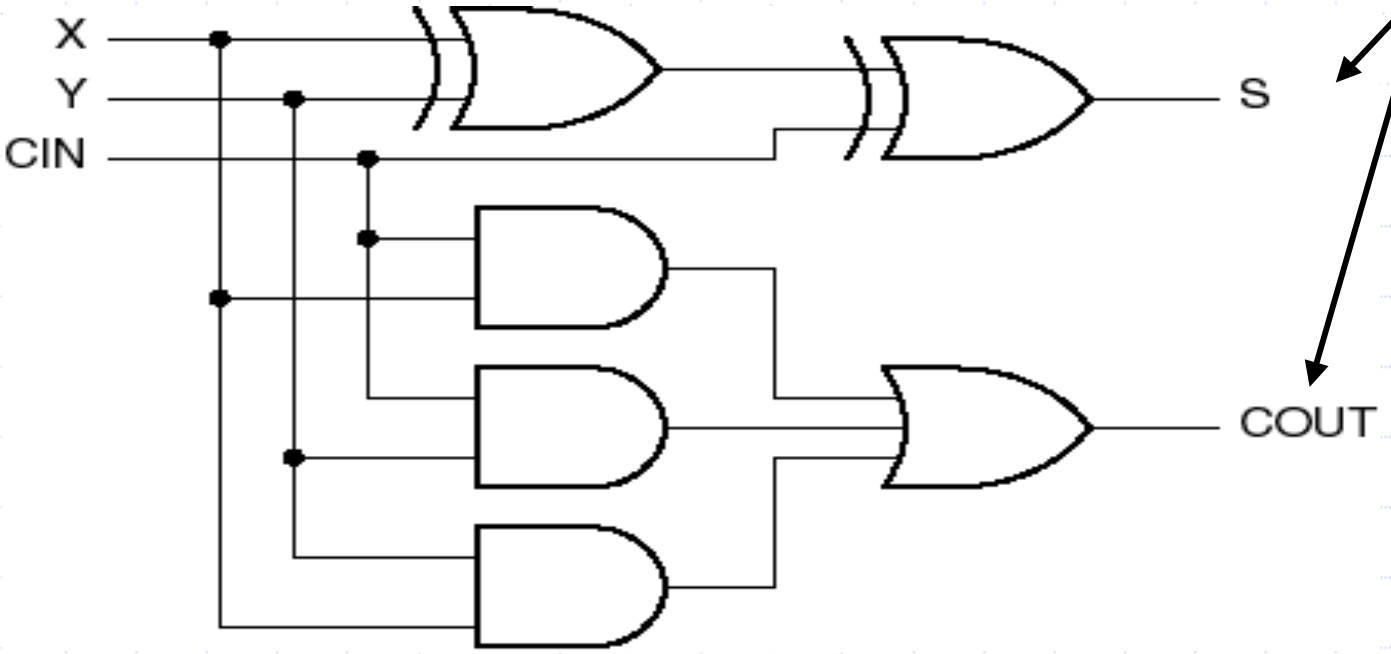
Full-adder circuit



$S = x'y'z + x'yz' + xy'z' + xyz$

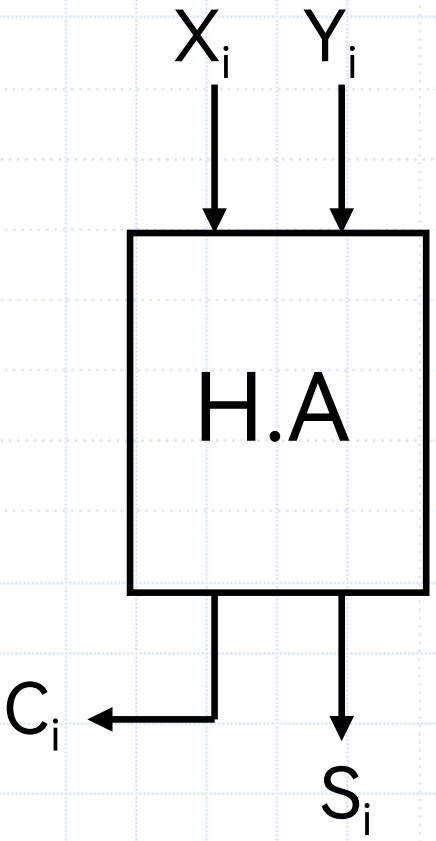


$S = xy + xz + yz$
$= xy + xy'z + x'yz$



2 gate delays

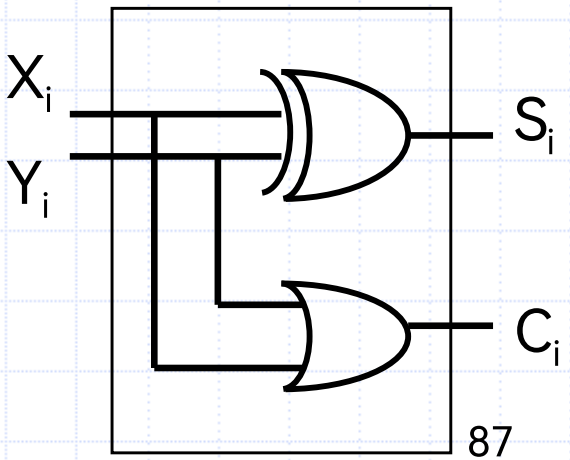
بلوک دیاگرام (H.A)



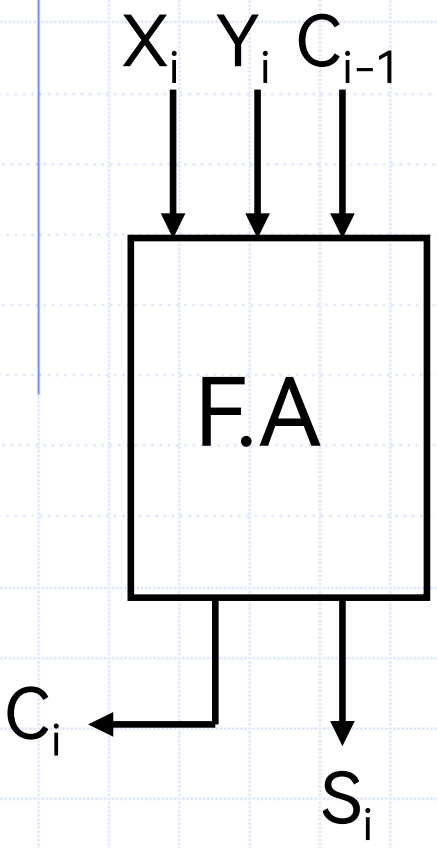
Truth Table

X_i	Y_i	C_i	S_i
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$\begin{cases} S_i = X_i \oplus Y_i \\ C_i = X_i Y_i \end{cases}$$



بلوك دياگرام (F.A)



Truth Table

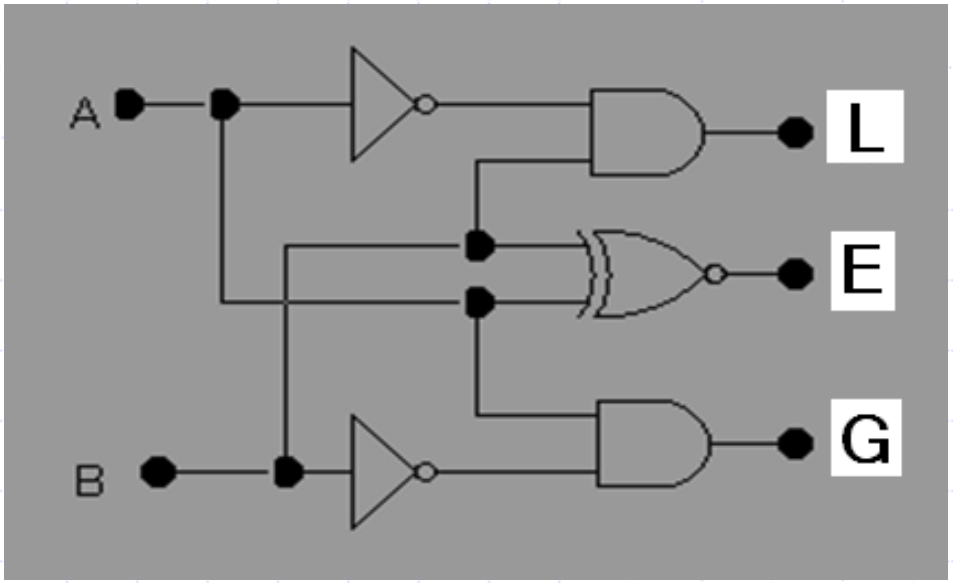
X_i	Y_i	C_{i-1}	C_i	S_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$\begin{cases} S_i = X_i \oplus Y_i \oplus C_{i-1} \\ c_i = X_i Y_i + X_i C_{i-1} + Y_i C_{i-1} \end{cases}$$

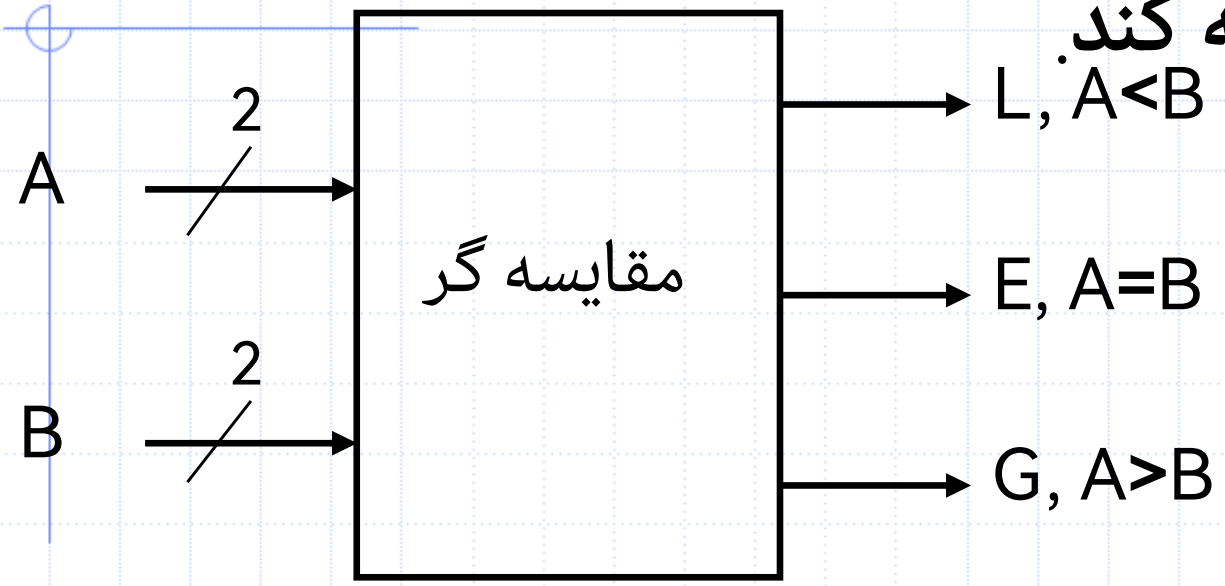
مقایسه گر (comparator) -

- مقايسه گر قطعه اي محاسباتي است كه اندازه نسبي دو عدد دودويي را معين مي كند.
- در يك مقايسه گر، سه تصميم در مورد دو كلمه انجام و در خروجي ها قرار مي گيرند. يعني A=B , A>B , A<B اگر

A	B	G	L	E
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	0	0



- مقایسه گري طراحي کنيد که دو کلمه $A=(A_1A_0)_2$ و $B=(B_1B_0)_2$ را در کد دودويي مقايسه کند.



L=1, If A<B
E=1, If A=B
G=1, If A>B

$L= A_1 \overline{B_1} + A_1 \overline{A_0}\,\overline{B_0} + A_0 \overline{B_1}\,\overline{B_0}$ For $(A_1A_0)_2 < (B_1B_0)_2$

$E=\overline{A_1}\,\overline{A_0}\,\overline{B_1}\,\overline{B_0} + \overline{A_1}A_0\,\overline{B_1}B_0 + A_1\overline{A_0}\,\overline{B_1}\,\overline{B_0} + A_1A_0B_1B_0$ For $(A_1A_0)_2 = (B_1B_0)_2$

$G=A_1\overline{B_1} + A_1\overline{B_1}\,\overline{B_0} + A_1A_0\overline{B_0}$ For $(A_1A_0)_2 > (B_1B_0)_2$

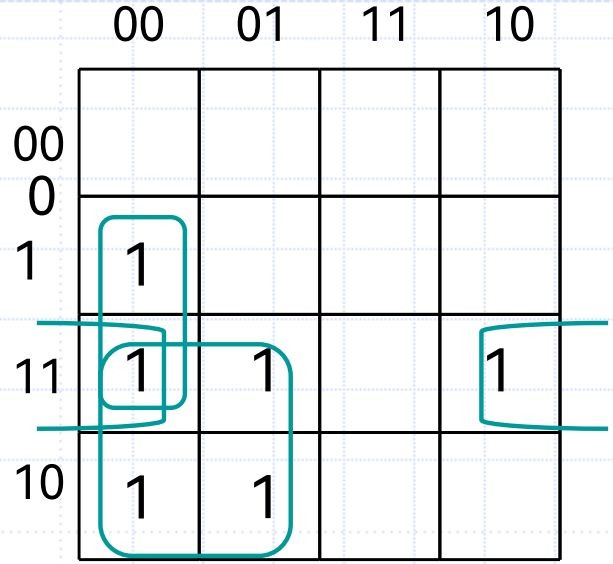
$B=(B_1B_0)_2$ $A=(A_1A_0)_2$

- برای مقایسه A و B باید از بیت های با ارزش مقایسه را انجام داد.
- A وقتی از B بیشتر است که ($A_1=1$ و $B_1=0$) یا اینکه اگر $B_1=A_1$ آنگاه ($B_0=0$و $A_0=1$) و به همین ترتیب.

A_1	A_0	B_1	B_0	L	E	G
0	0	0	0	0	1	0
0	0	0	1	1	0	0
0	0	1	0	1	0	0
0	0	1	1	1	0	0
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	0	1	0
1	0	1	1	0	0	1
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	0	1	0

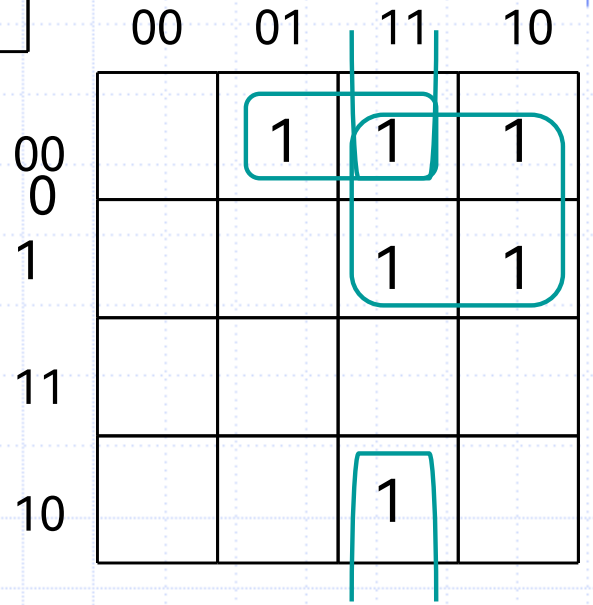
$$F_2=\bar{A}_1\bar{A}_0B_1B_0+\bar{A}_1A_0\bar{B}_1\bar{B}_0+A_1\bar{A}_0\bar{B}_1B_0+A_1A_0B_1B_0$$

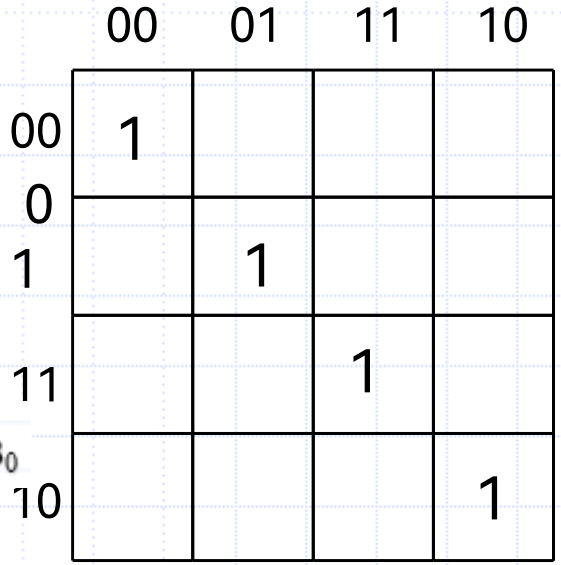
E, A=B



G, A>B

$$F_3=A_1\bar{B}_1+A_1B_1\bar{B}_0+A_1A_0\bar{B}_0$$





L, A<B

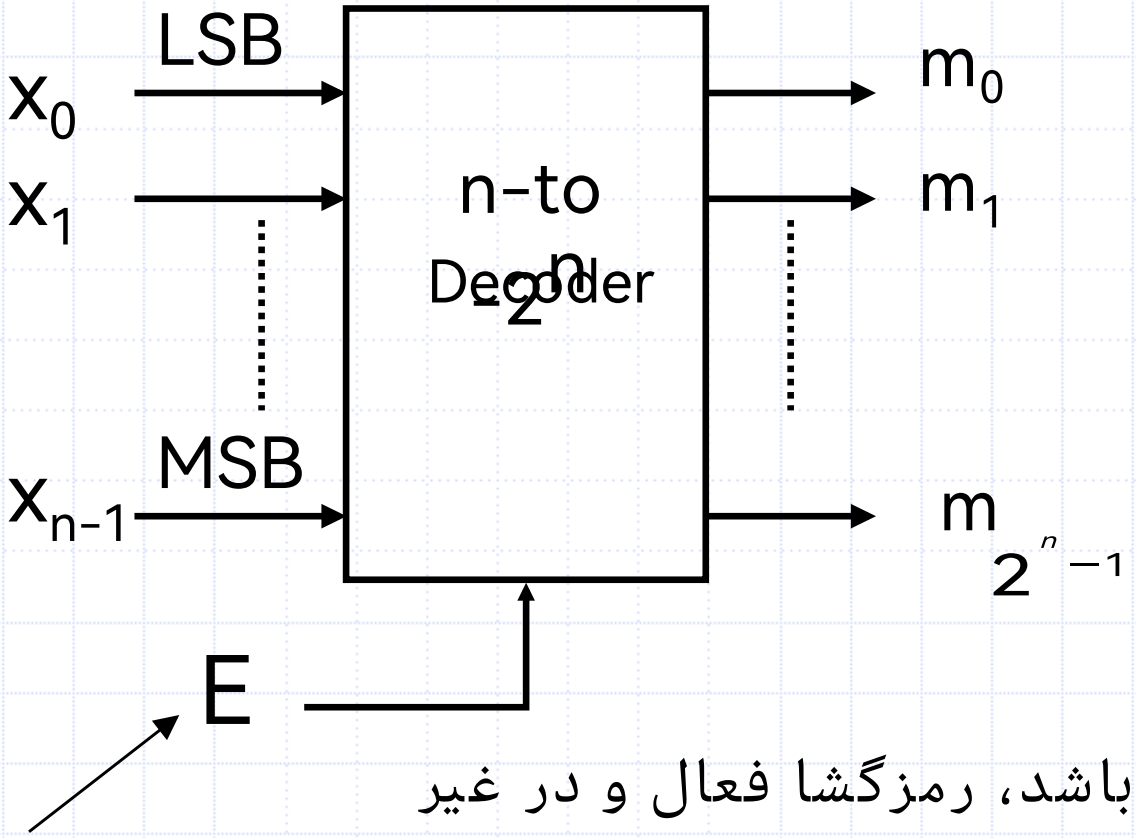
$$F_1=A_1\bar{B}_1+A_1\bar{A}_0\bar{B}_0+A_0\bar{B}_1B_0$$

رمزگشا (Decoder) -

- دیکدر یک مدار ترکیبی است که اطلاعات باینری حاصل از n ورودی را به 2^n خروجی مجزا تبدیل می کند.
- به آن مدار کشف کننده رمز یا رمزگشا هم مي‌گويند. داراي n ورودي و حداکثر 2^n خروجي مي‌باشد.
- به ازاي هر ورودي فقط و فقط يكي از خروجي‌ها فعال است.
- **کاربردها:** رمزگشا در پیاده سازی توابع، آدرس‌دهی خانه‌**های حافظه** و تبدیل کدها به یکدیگر کاربرد دارد.
- هر خروجی دیکدر معادل با یک مینترم از متغیرهای ورودی است. (تولید کننده مینترم)
- **عنصري است که مینترم ها را مي سازد.**
- مثلاً رمزگشای 3 به 8
  - سه خط ورودی که مقادیر 000 تا 111 را **می‌پذیرند**.
  - **به**ازای هر یک از ترکیبات ممکن برای ورودی‌ها یک خط خروجی داریم (8 خط خروجی)

دیکدر n به 2^n یا دیکدر ماجول

- دیکدر n به 2^n یك مدار منطقي تركيبي است با n خط ورودي و حداكثر 2^n سيگنال خروجي منحصربفرد.



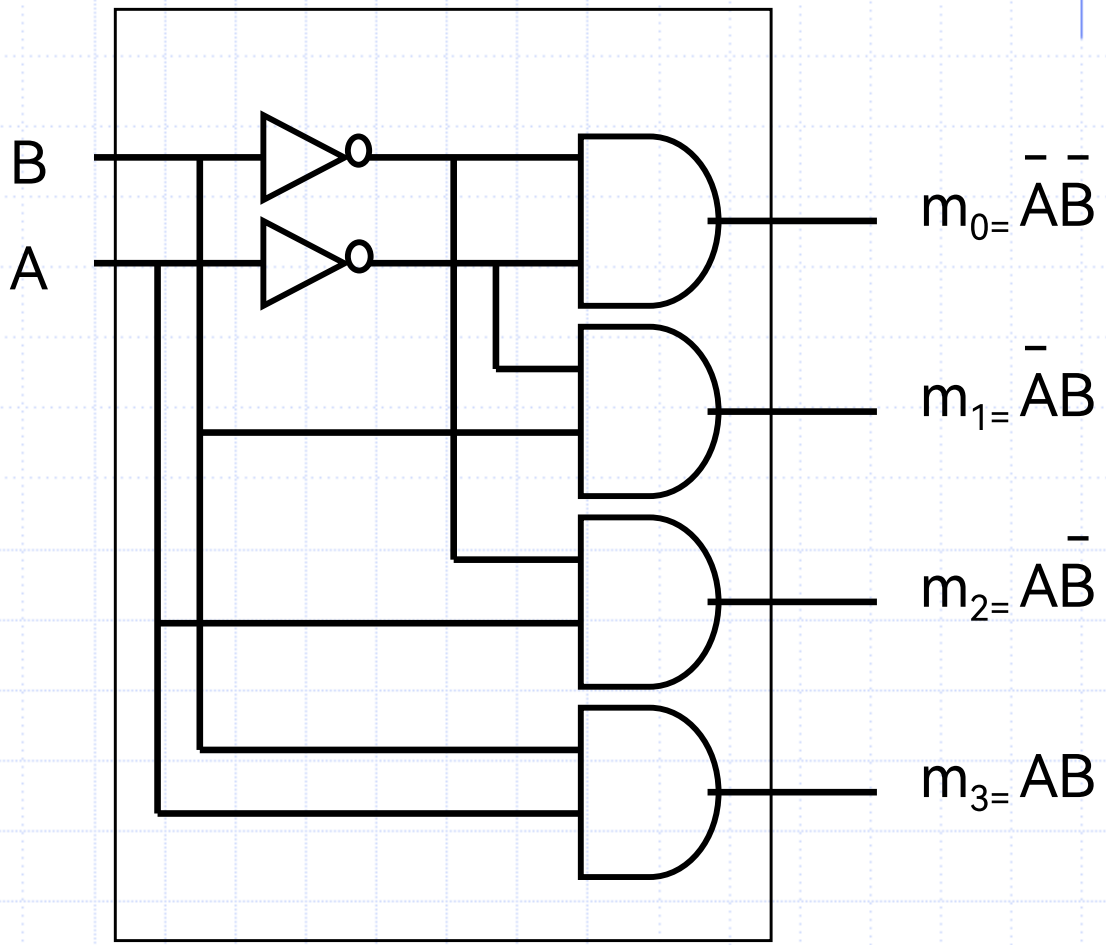
- یک یا چند ورودی فعال ساز
  - فعال بالا: اگر E=1 باشد، رمزگشا فعال و در غیر اینصورت غیر فعال است.
  - فعال پایین: اگر E=0 باشد، رمزگشا فعال و در غیر اینصورت غیر فعال است.

معمولا Active Low هستند.

دیاگرام منطقي (موازي و خروجي هاي فعال بالا)

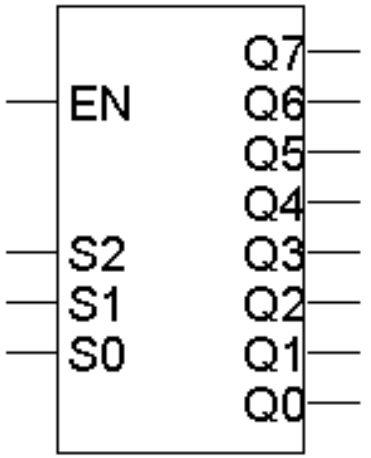
Truth Table

E	A	B	m_0	m_1	m_2	m_3
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1
0	×	×	0	0	0	0

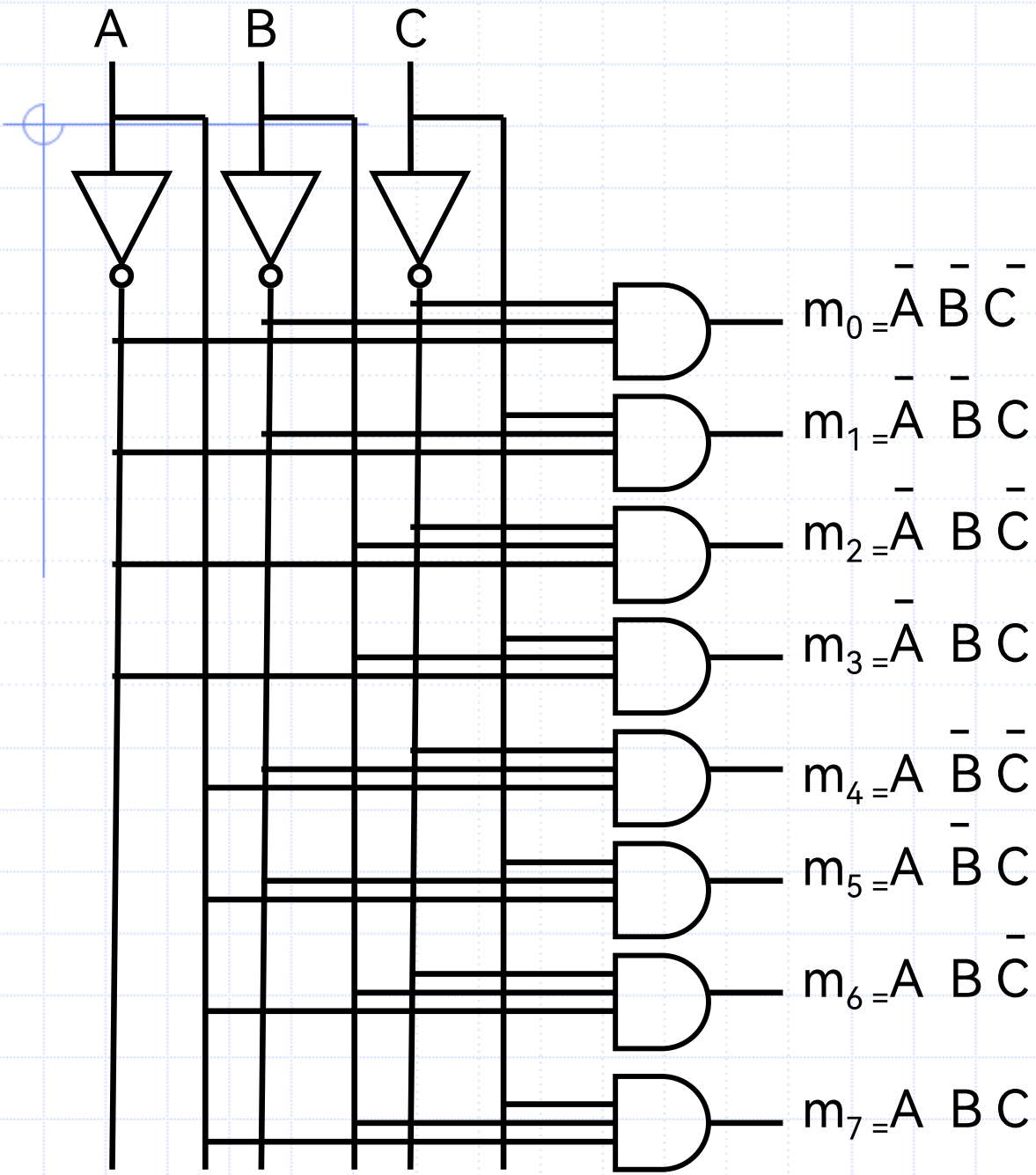


جدول صحت رمزگشای 3 به 8

S 2	S 1	S 0	Q 0	Q 1	Q 2	Q 3	Q 4	Q 5	Q 6	Q 7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1



ـ دیکدر نوع موازي سه بيت(3 به 8)

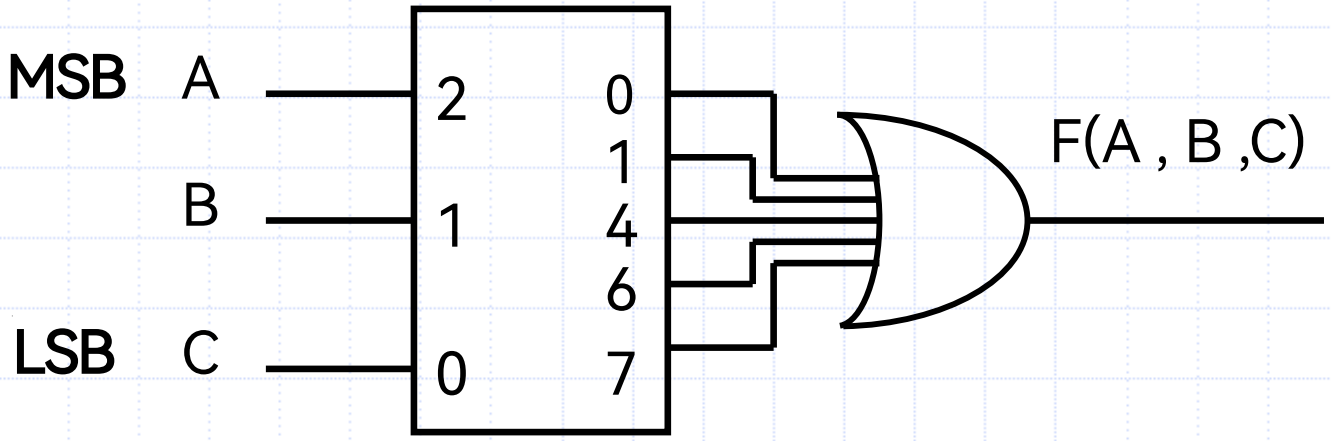


پیاده‌سازي توابع بولي با استفاده از دیکدر

- دیکدرها در واقع تمامي جملات مینیمم (مینترمها) را تولید مي‌کنند. از طرف دیگر هر تابع بولي را مي‌توان بصورت مجموعي از مینترمها نشان داد.
- لذا از ترکیب یك دیکدر و یك گیت OR مي‌توان هر تابعي را ساخت.
- با استفاده از این روش هر مدار ترکیبی با n ورودی و m خروجی می تواند بوسیله یک دیکدر n به 2^n و m گیت Or پیاده سازی شود.

یك دیکدر (با خروجي فعال بالا) و یك گیت ORبکار بریم.

$$F(A , B , C) = m_0 + m_1 + m_4 + m_6 + m_7$$

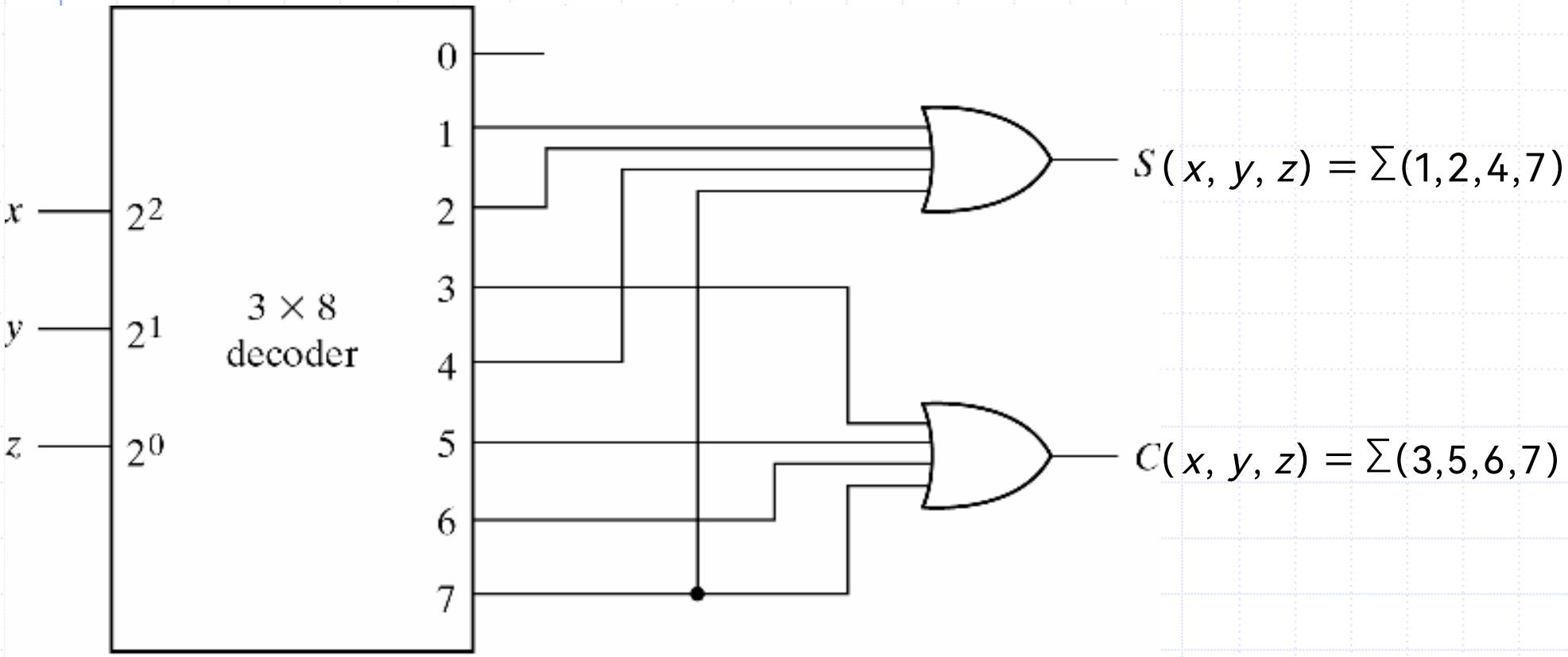


ساخت تمام جمع کننده با دیکدر

- مداری است با سه ورودی، روابط مربوط به تمام‌جمع‌کننده:

$S(x,y,z)=\Sigma(1,2,4,7)$

$C(x,y,z)=\Sigma(3,5,6,7)$



رمزکننده (Encoder) –

- به آن رمزکننده نیز می گویند.
- اینکدر يك ماجول تركيبي است كه براي هر سيگنال ورودي به دستگاه يك كد خروجي منحصر به فرد را اختصاص مي‌دهد.
- اگر يك ماجول اينكدر n ورودي داشته باشد تعداد خروجي s بايد در رابطه زير صدق كند:

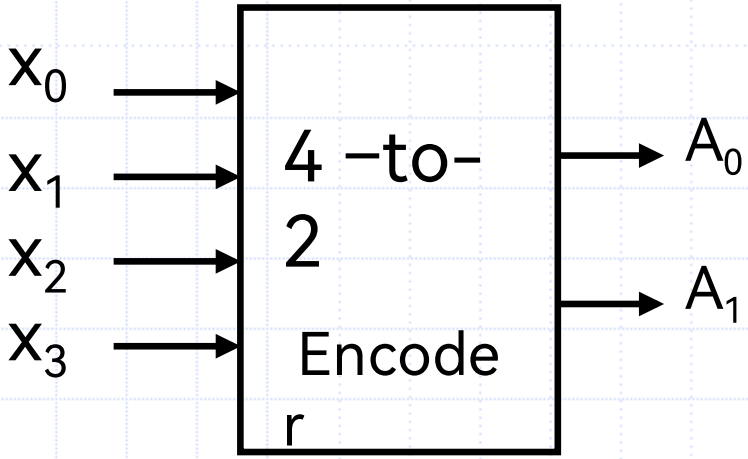
$2^s \geq n$

$s \geq \text{Log}_2\ n$

- مداري **ديجيتالي است كه** حداكثر 2^n ورودي و n خط خروجي دارد. خروجی، آدرس ورودی فعال را مشخص می کند.

- رمزگذار (Encoder)

- خطوط خروجی، معادل دودوئی ورودی را نشان می دهند.
- فقط یکی از ورودی ها در هر لحظه می تواند یک (فعال) باشد.
  - در غیر این صورت مدار بی معنی می باشد.
- **يك اينكدر با چهار خط ورودي (بشرطي كه در هر لحظه از زمان فقط يك ورودي فعال باشد.)**



جدول ارزش یک رمزگذار 8 به 3 -

X_7	X_6	X_5	X_4	X_3	X_2	X_1	X_0	A_2	A_1	A_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

$$A_0=X_1+X_3+X_5+X_7$$
$$A_1=X_2+X_3+X_6+X_7$$
$$A_2=X_4+X_5+X_6+X_7$$

- فرض بر **اینست که در هر زمان فقط یک خط ورودی فعال** است. در صورتیکه بیش از یک ورودی فعال باشد، خروجی انکدر نادرست است.

تسهیم کننده (Multiplexer) -

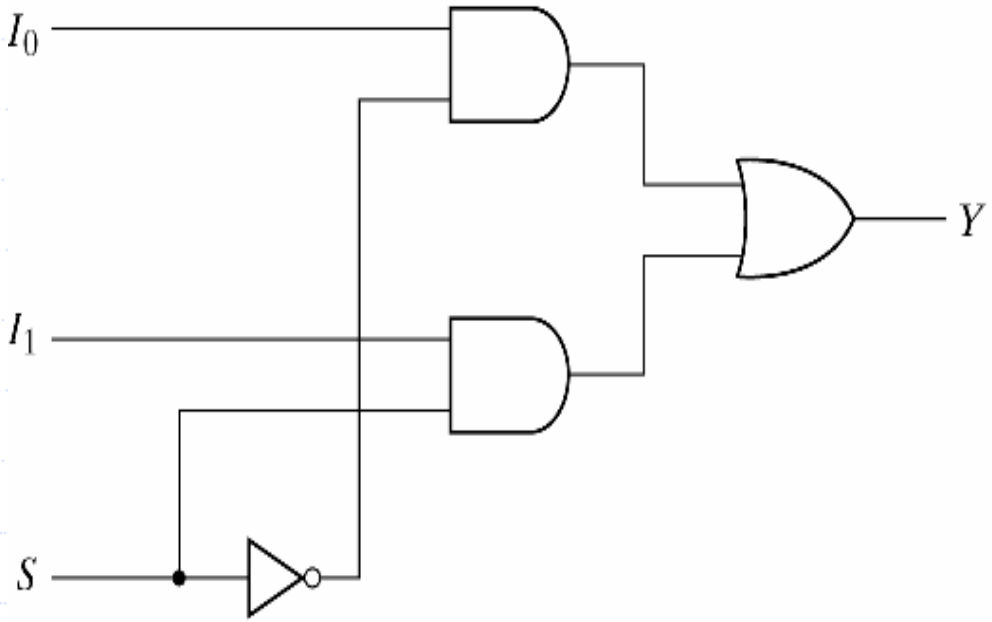
- یک مدار ترکیبی است با 2^n خط ورودی، یک خط خروجی و n خط ورودی انتخاب. مقادیرمختلف خطوط انتخاب مشخص می‌کند که کدام یک از خطوط ورودی به سمت خروجی هدایت شود.
- بعبارت دیگر، انتخاب يك خط ورودي توسط مجموعه‌اي از خطوط انتخاب كنترل مي‌شود.

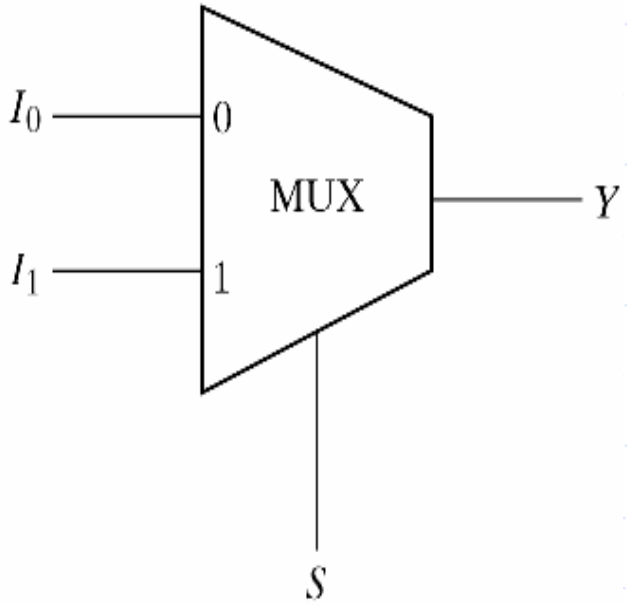
- می توان گفت که **خروجی مالتی** پلکسر، **تابعی از** ورودی‌ها و خطوط انتخاب است.

- از مالتی پلکسر در پیاده سازی توابع، اتصال یک منبع به چند استفاده کننده و اتصال چند منبع به یک استفاده کننده، استفاده می شود.

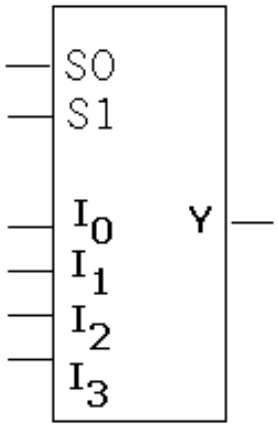
2 – MUX به 1

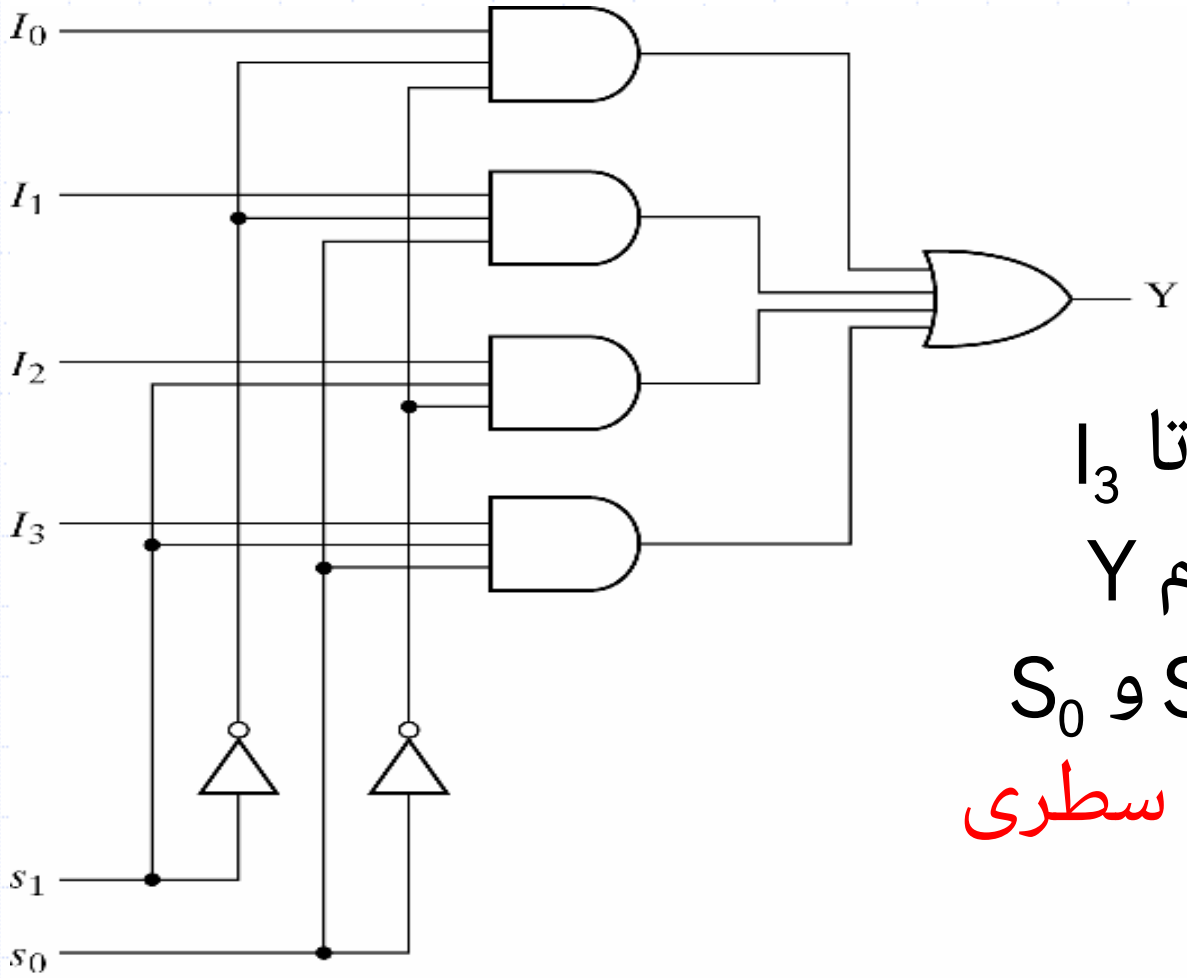
- منظور از S همان ورودي انتخاب است. I_0 و I_1 نيز دو خط ورودي هستند.





MUX 4x1 (مدار داخلي) -





جدول عملکرد

s_1	s_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

- 4 خط ورودی I_0 تا I_3
- یک خروجی با نام Y
- 2 خط انتخاب S_1 و S_0
- جدول عملکرد 4 سطری

پیاده سازی توابع بولی توسط مولتی پلکسرها

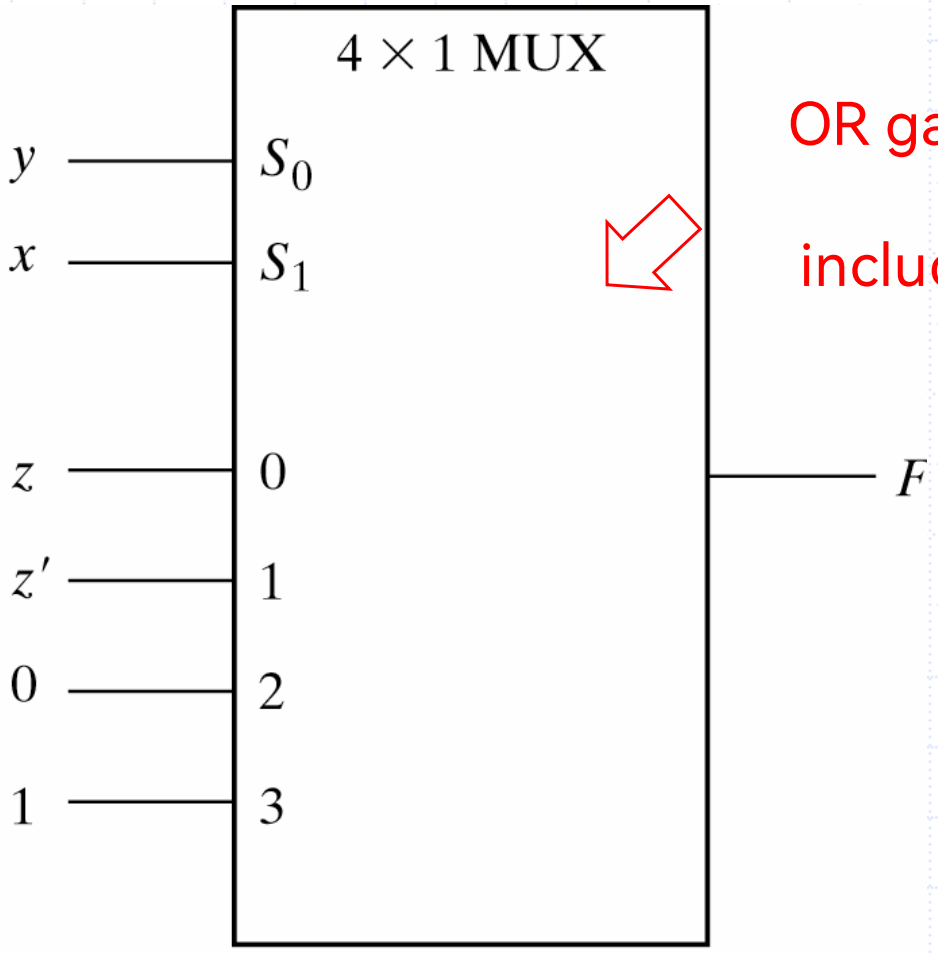
- هر تابع n+1 متغیره را می توان توسط یک مولتی پلکسر که n خط انتخاب دارد ساخت. برای این کار:
  - ساده‌تر اینست که آخرین متغیر را بعنوان متغیر منفرد انتخاب کرده، بقیهٔ **متغیرها را به ورودي‌هاي انتخاب**MUX بدهیم.
  - ابتدا جدول درستی را تشکیل دهید. جدول درستي را از بالا دو سطر، دو سطر جدا کنید.
  - به ازای هر ترکیب از ورودیهای انتخاب، خروجی را بر حسب ورودی باقیمانده (کم ارزشترین ورودی) تعیین کنید. اگر A متغیر منفرد (باقي‌مانده) تابع بولي باشد، ورودي‌هاي MUX مي‌توانند A، A′، 1 و یا 0 باشند.

  - صفر هستند، آنگاه ورودي متناظر از MUX برابر با 0 قرار داده مي‌شود.
  - 1 هستند، آنگاه ورودي متناظر از MUX برابر با 1 قرار داده مي‌شود.
  - مقادير متغير منفرد هستند، آنگاه همان متغير منفرد به ورودي متناظر از MUX داده مي‌شود.
  - مکمل مقادير متغير منفرد هستند، آنگاه مکمل متغير منفرد به ورودي متناظر از MUX داده مي‌شود.

- جدول درستي تابع بولي زير همراه با نحوة پيادهسازي آن توسط MUX نشان داده شده است:

$$F(x, y, z) = \sum(1,2,6,7)$$

x	y	z	F	
0	0	0	0	$F = z$
0	0	1	1	
0	1	0	1	$F = z'$
0	1	1	0	
1	0	0	0	$F = 0$
1	0	1	0	
1	1	0	1	$F = 1$
1	1	1	1	

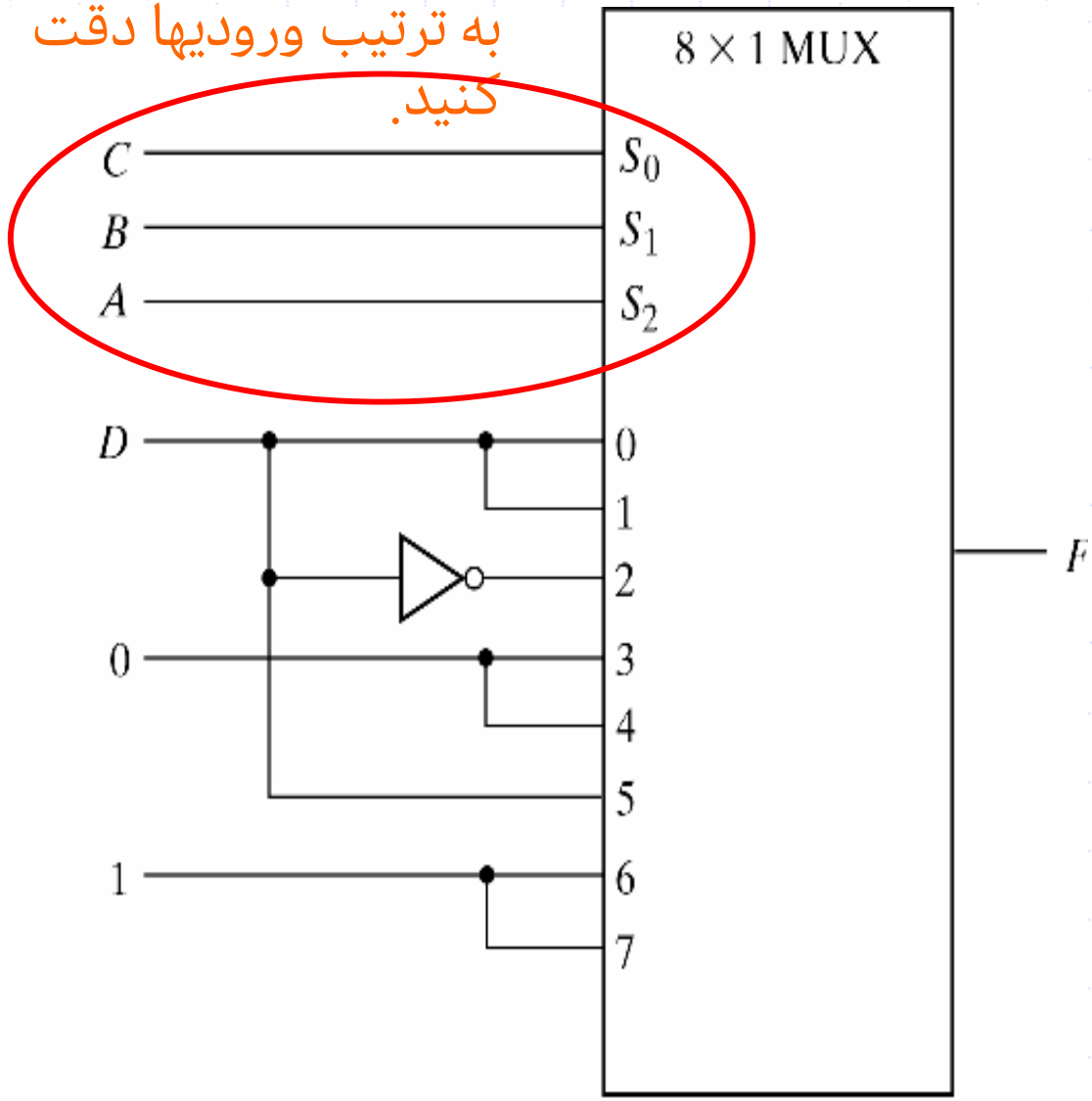


- جدول درستي تابع بولي چهار متغيره زير همراه با نحوة پيادهسازي آن توسط MUX نشان داده شده است:

$$F(A, B, C, D) = \sum(1,3,4,11,12,13,14,15)$$

A	B	C	D	F	
0	0	0	0	0	$F = D$
0	0	0	1	1	
0	0	1	0	0	$F = D$
0	0	1	1	1	
0	1	0	0	1	$F = D'$
0	1	0	1	0	
0	1	1	0	0	$F = 0$
0	1	1	1	0	
1	0	0	0	0	$F = 0$
1	0	0	1	0	
1	0	1	0	0	$F = D$
1	0	1	1	1	
1	1	0	0	1	$F = 1$
1	1	0	1	1	
1	1	1	0	1	$F = 1$
1	1	1	1	1	

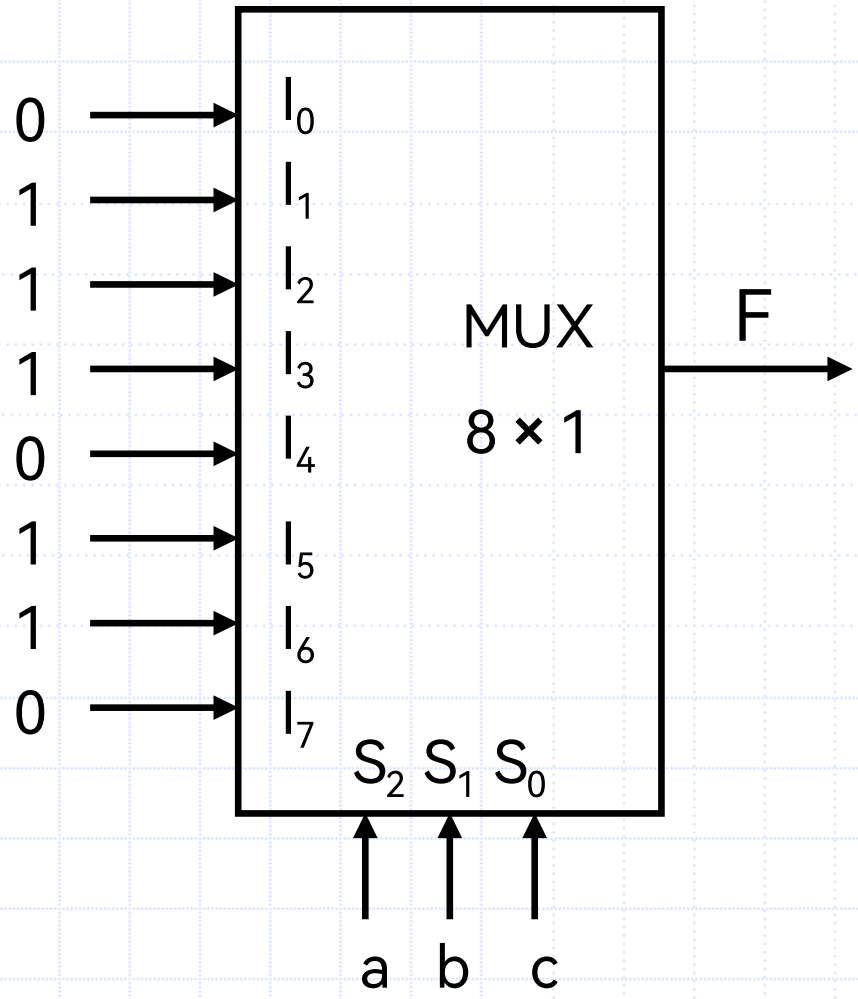
به ترتیب ورودیها دقت کنید.



- جدول درستي تابع بولي زير همراه با نحوة پيادهسازي آن توسط MUX نشان داده شده است:

$$F(A,B,C) = \sum(1,2,3,5,6)$$

a	b	c	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0



- جدول درستي تابع بولي زير همراه با نحوهٔپياده‌سازي آن توسط MUX نشان داده شده است: (روش ديگر)

$$F(A, B, C) = \sum(1,2,3,6)$$

	a	b	c	F
I_0	0	0	0	0
	0	0	1	1
I_1	0	1	0	1
	0	1	1	1
I_2	1	0	0	0
	1	0	1	0
I_3	1	1	0	1
	1	1	1	0

